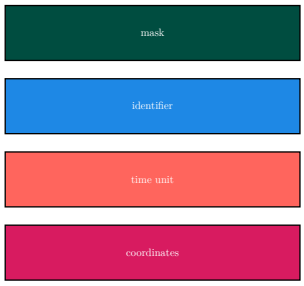
Graphical Abstract

readcon-core: a versioned geometry checkpoint for resumed saddle searches

Rohit Goswami



Highlights

readcon-core: a versioned geometry checkpoint for resumed saddle searches

Rohit Goswami

- CON 3.1 states the eOn saddle-search checkpoint as a versioned text grammar.
- The format defines per-component masks, atom identifiers and velocity time units.
- Two corpora of legal and illegal files decide what a conforming reader accepts.
- A climbing-image band resumed from CON 3.1 repeats the uninterrupted run bit for bit.
- Read times are measured against ASE, chemfiles, h5py and native trajectory readers.

readcon-core: a versioned geometry checkpoint for resumed saddle searches

Rohit Goswami[a,b,c,1]

[a] *TurtleTech ehf., Fornhaga 24, 107, Reykjavík, Iceland*
[b] *Science Institute and Faculty of Physical Sciences, University of Iceland, VR-III, 107, Reykjavík, Iceland*
[c] *SURF B.V., Science Park 140, 1098 XG, Amsterdam, The Netherlands*

Abstract

CON is a geometry file format used in many codes. readcon-core formalizes that text as CON 3.1 and improves the fields a restart reads back. A JSON line states the version and the units. Each coordinate row carries a mask of fixed components and an atom identifier, and a reader of a known major accepts any later minor. Two corpora of 165 files fix the verdict of a conforming reader, and every Python and C read path returns that verdict. On a platinum slab, a climbing-image band resumed from CON 3.1 repeats the uninterrupted band, and a lost hold changes the prefactor by a factor 37. A borrowed DLPack view of 32768 positions costs 0.07908 times an explicit copy, the median of eight repeats.

Keywords: geometry checkpoint, file format specification, saddle search, conformance corpus, nudged elastic band

PROGRAM SUMMARY

Program Title: readcon-core
CPC Library link to program files: (to be added by Technical Editor)
Developer's repository link: https://github.com/lode-org/readcon-core
Code Ocean capsule: (to be added by Technical Editor)
Licensing provisions: MIT
Programming language: Rust, with C, C++, Python, Julia and Fortran 2008 interfaces

Email address: rgoswami@ieee.org (Rohit Goswami)
[1] ORCID: 0000-0002-2393-8056.

Supplementary material: the normative specification, the two conformance corpora and the scripts and data behind every table and figure

Nature of problem: Rare-event campaigns stop and resume saddle searches from a stored geometry. A resumed surface search reads which Cartesian components of each atom are fixed, because a held component removes a mode from the harmonic prefactor and can move the saddle. It reads an identifier on each row, because events and band images are matched by atom identity, and the time unit of the velocities, because a misread unit scales the kinetic energy by the square of the conversion factor. A restart that repeats the interrupted run to roundoff also reads back the binary64 coordinates as written. Of the formats in common use, H5MD defines no mask element, the POSCAR structure file defines no atom identifier, and extended XYZ keeps both in column names that one library chose and names no time unit. CON is a geometry format used in many codes, and it had no specification, so each reader decided for itself which files are legal and what each column means.

Solution method: CON 3.1 is a versioned grammar for that text, and readcon-core is the reference reader and writer that formalizes and improves it. The second line of a frame is a JSON object that declares the major and minor version, the units, the periodic flags and the per-atom sections after the coordinates. The fourth column of a coordinate row is a three-bit mask of fixed components and the fifth an atom identifier. From minor 1 a velocity section declares its time unit. A reader of a known major accepts any later minor, keeps unknown keys and skips a later section by its declared width. The writer prints the shortest decimal that reads back as the same binary64 value, and the reader rounds each decimal to the nearest binary64 value. Two corpora of legal and illegal files fix the verdict of a conforming reader on the clauses of the specification. readcon-core implements the grammar once in Rust, behind a C interface for C++, Julia and Fortran and a PyO3 binding for Python. On more than one thread it walks the frame boundaries serially by the counts in each header, parses the frames in parallel and returns them in file order. A path writer replaces the destination only after the last frame is accepted.

Additional comments including restrictions and unusual features: The release described is readcon-core 0.18.1 (Data availability). Every build reads gzip input; zstd input and parallel parsing are build features, and the Python wheel enables both. A frame stores Cartesian coordinates, the cell, the periodic flags, the mask, one identifier per atom, and optional per-atom velocities, forces, energies, charges, spins, magnetic moments, displacements and spreads. It stores no optimiser history, so an optimiser with memory takes a different path after a restart. Files that earlier eOn releases wrote are grammar version 1 and read as such. The velocities of a file of an earlier version are read in the natural unit of eOn whatever time unit the file names. In Python, `readcon.read_con(path)` returns the frames of a file

(Listing 2).

References: eOn [1, 2]; readcon-core releases [3, 4].

1. Introduction

Rare-event methods turn a potential energy surface into rates. Transition state theory [5, 6, 7] reduces the kinetics of an activated process to its saddle points and the vibrations there. The saddles are located by the nudged elastic band [8, 9, 10, 11], by minimum-mode followers [12, 13, 14] and by string methods [15, 16]. Adaptive kinetic Monte Carlo (AKMC) and accelerated dynamics chain them into trajectories of seconds and longer [17, 18, 19, 20]. A long-timescale run in eOn [1, 2] spawns thousands of saddle searches, often on volunteer or distributed resources [21, 22]. It passes their geometries between Python drivers [23, 24], C++ engines and Fortran electronic structure codes [25]. A search that stops on one machine resumes on another from a geometry file, the checkpoint of the search.

The checkpoint has to return four fields that a trajectory reader can ignore. A surface search fixes the slab and can hold an adatom at a set height while it crosses the surface, so mobility belongs to each Cartesian component, and a lost hold changes the saddle that the search converges to (Section 7.2). Mode following and AKMC bookkeeping match atoms across frames and events by an identifier on each row. A reader has to return that identifier as written rather than as the row order. Dynamical steps read velocities, and the natural time unit of eOn is $(\mathrm{amu}\,\mathrm{Å}^2/\mathrm{eV})^{1/2} = 10.18\,\mathrm{fs}$, so a reader that takes the same numbers as Å/fs scales the kinetic energy by 103.64. A restart that repeats the interrupted run to roundoff reads back binary64 coordinates [26]. For that, the writer prints the shortest decimal that reads back as the same double [27, 28], and the reader rounds each decimal to the nearest double [29]. Two readers given the same bytes must then return the same fields, and that takes a versioned grammar rather than the convention of one reader.

Existing formats keep some of these fields. Extended XYZ adds key-value metadata to the comment line of the plain Cartesian coordinate format (XYZ) and publishes a grammar [30]. It keeps masks and identifiers in column names that the Atomic Simulation Environment (ASE) chose [23, 31], and a file declares no version and no velocity unit. H5MD [32, 33, 34] stores identity, velocities with units and the box in HDF5, and it defines no element for a per-component constraint. The GROMACS compressed trajectory (XTC) and full-precision trajectory (TRR) formats [35], the binary trajectory (DCD)

format of CHARMM [36] and the trajectory next generation (TNG) format [37, 38] store molecular dynamics output with neither a mask nor a velocity unit. The ASE trajectory keeps every field in a layout that the Python code of ASE defines. POSCAR, the structure file of the Vienna Ab initio Simulation Package [39, 25], stores a per-component hold as selective-dynamics flags but no atom identifier. Many of these formats are read and written by chemfiles [40, 41], MDAnalysis [42, 43], the Open Visualization Tool (OVITO) [44] and Open Babel [45]. Of the four, none opens a version-1 CON file or a CON 3.1 file, and the eon module of ASE opens both but stops at the velocity section of the frame of Section 2.

Most atomistic formats have no specification apart from the code that reads them, so that code is the de facto definition of the format. Particle physics fixed the files that pass between independent event generators in published texts: the Les Houches event file [46], the second Les Houches Accord for supersymmetric spectra [47] and the HepMC3 library for generator events [48]. Electronic structure has done the same for densities and wavefunctions with the European Theoretical Spectroscopy Facility (ETSF) file format [49] and TREXIO [50], and for pseudopotentials with the pseudopotential markup language (PSML) [51]. A computed barrier is reproducible from a file only when every reader returns the same fields from it. A versioned specification and a corpus of files with known verdicts turn that condition into a test of any reader.

Here, we describe CON 3.1 and readcon-core. CON is a file format used in many places. CON 3.1 formalizes that text, and readcon-core improves it with a JSON header line that declares a version, the units and the periodic flags, a three-bit mask of fixed components and an atom identifier on each coordinate row, declared per-atom sections, and coordinates that read back as the binary64 values written. A borrowed DLPack view of the positions costs 0.07908 times an explicit copy (Section 5). readcon-core is its reference reader and writer, one Rust core behind C, C++, Python, Julia and Fortran interfaces, a layout that it takes from chemfiles [40]. Section 2 reads one frame line by line and measures what each field changes when a reader gets it wrong, and Section 3 states the grammar and its version rule. Sections 4 and 6 give the reader and test it against the corpora, Section 7 resumes saddle searches from checkpoints that we wrote and read back with readcon-core and with other libraries, and Section 8 times readcon-core against ASE, MDAnalysis, OVITO, pymatgen and a pure Python parse of the same CON file. Unlike H5MD, TNG and extended XYZ, the CON 3.1 specification defines all four

fields that a restart reads, and its corpus of legal and illegal files decides the verdict of any reader on each clause. readcon-core is the only reader in this comparison that returns the fixed-component mask, the atom identifier and the velocity time unit from places that specification defines. No other reader both accepts a later minor of a known major and decides each file of the conformance corpus.

2. One frame, line by line

Listing 1 is a complete CON 3.1 file, the survey frame of a Pt(100) slab of two layers of four atoms with one adatom, written by readcon-core with velocities. The left panel of Figure 1 shows the slab, and Table 1 gives the content of each line and the rule that reads it. We read the fields in file order, and at each we give what a reader returns when it gets that field wrong.

Listing 1: A Pt(100) slab of two layers of four atoms with one adatom, as one CON 3.1 frame with velocities written by readcon-core 0.17.4. Section 2 reads it line by line.

```

{"con_spec_minor":1,"con_spec_version":3,"pbc":[true,true,false],"sections":["velocities"],"units":{"energy":"eV","length":"angstrom","mass":"amu","time":"(amu*angstrom^2/eV)^0.5"}}
5.5437171645025325 5.5437171645025325 13.96
90.0 90.0 90.0


1
9
195.084
Pt
Coordinates of Component 1
1.386493891622963 1.3858827980505872 5.999231933623632 7 17
4.158528117245317 1.3861213257350742 5.999813419553613 7 3
1.3860627284818523 4.158248487145931 6.000055839372117 7 11
4.1574916113945894 4.157781657328241 6.000983049344183 7 5
-0.0002684564058587828 0.0006114378220263555 7.959946638798572 0 23
2.7723693731828494 0.0002842894752666141 7.959407544708384 0 2
0.000771950084012911 2.772315330818269 7.960901835964954 0 13
2.772542023981585 2.7723297461317795 7.959545089014511 0 7
1.3867197337624881 1.3861370701026228 9.760314828133515 4 41

Pt
Velocities of Component 1
0.0 0.0 0.0 7 17
```

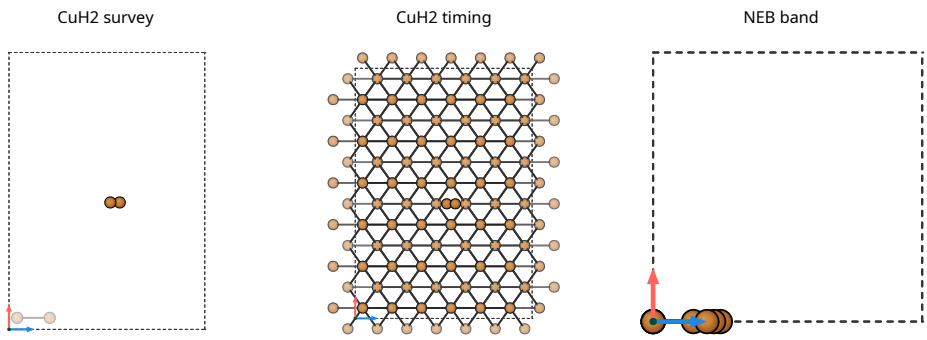


Figure 1: **The three geometries behind the measurements.** Left: the survey frame of Listing 1 and Table S3, a Pt(100) slab with one adatom. Middle: the CuH_2 slab used as a structure in the measurements. Right: the middle image of the uninterrupted climbing-image band of the EMT Pt(100) adatom hop of Section 7. The colours are the Ruhi palette.

```
0.0 0.0 0.0 7 3
0.0 0.0 0.0 7 11
0.0 0.0 0.0 7 5
-0.004419216359135741 -0.0031227306022671786 0.0053252274765567134 0 23
-0.0019411746848912532 -0.004423003635856234 0.004825314186834311 0 2
-0.00572219106757806 -0.003294382313545619 -0.00037362717028046195 0 13
-0.0018722607532418165 0.0008120539032663098 0.002918400292950336 0 7
-0.0016020339068388144 0.004319885827377455 0.0 4 41
```

The header line is one JSON object [52], recognised by its first non-blank character, `{`, and a frame whose header is anything else is grammar version 1, the text that earlier eOn releases write. The object of Listing 1 names the major and the minor of the grammar, declares the slab periodic in x and y and open in z, lists `velocities` as the one section after the coordinates, and states the units of length, energy, mass and time.

The two cell lines carry the lengths $a = b \approx 5.544\,\text{Å}$ and $c = 13.96\,\text{Å}$ and the angles α, β and γ in degrees, from which the reader rebuilds the cell with $\mathbf{a}$ along x and $\mathbf{b}$ in the xy plane [53],

$$\begin{aligned} \mathbf{a} &= a\,\hat{\mathbf{x}}, \qquad \mathbf{b} = b\cos\gamma\,\hat{\mathbf{x}} + b\sin\gamma\,\hat{\mathbf{y}}, \\ \mathbf{c} &= c\cos\beta\,\hat{\mathbf{x}} + c\,\kappa\,\hat{\mathbf{y}} + c\sqrt{\rho}\,\hat{\mathbf{z}}, \\ \kappa &= \frac{\cos\alpha - \cos\beta\cos\gamma}{\sin\gamma}, \qquad \rho = 1 - \cos^2\beta - \kappa^2. \end{aligned} \tag{1}$$

Table 1: The lines of Listing 1 and the rule that reads each.

Line	Content	Rule
1	Free text, empty	Kept on rewrite
2	Header: one JSON object with `con_spec_version` 3, `con_spec_minor` 1, `pbc`, `sections` and `units`	Any other header makes the frame version 1
3	Cell lengths a, b, c	Eq. (1)
4	Cell angles α, β, γ in degrees	Eqs. (1) and (2)
5, 6	Free, empty	Kept on rewrite
7	Atom types, 1	
8	Atoms of each type, 9	
9	Mass of each type, 195.084 amu	
10	Symbol, `Pt`	
11	`Coordinates of Component 1`	Labels compare case-insensitively
12--20	x, y, z, mask, identifier	Mask by Eq. (3): 7 on the 4 bottom rows, 0 on the top layer, 4 on the adatom
21	Blank	Opens the declared section
22	Symbol, `Pt`	
23	`Velocities of Component 1`	
24--32	v_x, v_y, v_z, mask, identifier	In the unit τ of Eq. (4)

In binary64 the cosine of the double nearest $\pi/2$ is 6.12×10^{-17}, so Eq. (1) gives this cell $b_x = 3.39 \times 10^{-16}$ Å and $c_x = c_y = 8.55 \times 10^{-16}$ Å. ASE 3.29.0 returns $b_x = 0$, because it sets the cosine of an angle within two units in the last place of a right angle to zero [23].

The radicand ρ decides whether three angles describe a cell at all. The volume of the cell of Eq. (1) is $V = a \cdot b \sin\gamma \cdot c\sqrt{\rho}$, and V^2 is the determinant of the Gram matrix G of the three vectors, so

$$\begin{aligned} \rho \sin^2\gamma = D &= 1 - \cos^2\alpha - \cos^2\beta - \cos^2\gamma + 2\cos\alpha\cos\beta\cos\gamma, \\ \det G &= a^2 b^2 c^2 D. \end{aligned} \tag{2}$$

No three vectors in space have angles with $D < 0$, and $D = 0$ is a flat cell. Two variants of the angle line show both cases. The exact ρ is zero for angles of 30, 60 and 90 degrees, since **c** then lies in the plane of **a** and **b**, and binary64 gives $\hat{\rho} = -2.22 \times 10^{-16}$. readcon-core accepts that frame with $c_z = 0$, because $\hat{\rho}$ lies inside the window of 1×10^{-12} that Section 3 gives for the roundoff of a zero radicand, and ASE stops on its own assertion that $\hat{\rho} \geq 0$. With 10, 10 and 170 degrees, $D = -3.82$ and $\rho = -127$. readcon-core rejects that frame with an error that names line 4, and ASE fails the same assertion without naming a line. ASE also refuses Listing 1 itself: its eon reader takes the blank line 21 for the first line of a second frame and stops when line 23 fails to parse as three cell lengths.

After two free lines come the counts, one atom type, nine atoms of that type and the mass of the type, and then a block for each type: its symbol, a label, and one row of x, y, z, the mask and the identifier per atom.

For a frame of major M, with $M = 1$ when the header is not JSON, the mask is the integer

$$m = \begin{cases} b_x + 2b_y + 4b_z, & M \geq 2, \\ b_x = b_y = b_z, & M = 1, \end{cases} \qquad b_\alpha \in \{0, 1\}, \tag{3}$$

with $b_\alpha = 1$ when component α is fixed. The bottom rows of Listing 1 are fixed in every component, the top layer is free, and the adatom is held in z alone. A reader that applies the version-1 rule of Table 2 to this frame rejects the mask of the adatom as an illegal flag, and one that applies the CON 3.1 rule to a version-1 file frees y and z of every atom held there. Section 7.2 measures what a lost hold of the adatom changes: the band crosses another saddle, with a larger prefactor.

The identifier is the index of the atom in the numbering of the writer, from 17 on the first row to 41 on the last, and between 2 and 41 in all. Grouping by type has permuted that numbering, and ascending identifier restores it. A reader returns the identifiers in file order and exactly, up to $2^{64} - 1$, and a row without an identifier takes its zero-based position in the frame. Of the cross-library round trips of Section 7, one returns no identifiers at all.

The blank line 21 opens the velocity section that the header declared, and the section repeats the block shape with one row of v_x, v_y, v_z, mask and identifier per atom. The held atoms have zero velocity and the adatom has $v_z = 0$, so 5 atoms move. Their unit is the one the header names, eOn's natural unit,

$$\tau = \left(\frac{m_\mathrm{u}\,\text{Å}^2}{\mathrm{eV}} \right)^{1/2} = \left(\frac{1.6605 \times 10^{-27}\,\mathrm{kg} \cdot 10^{-20}\,\mathrm{m}^2}{1.6022 \times 10^{-19}\,\mathrm{J}} \right)^{1/2} = 10.18\,\mathrm{fs}, \qquad (4)$$

with the CODATA 2018 values of the atomic mass constant m_u and the electronvolt [54]. In that unit the frame has a kinetic energy $\frac{1}{2}\sum_i m_i |\mathbf{v}_i|^2$ of 0.0177 eV. A reader that takes the same numbers as Å/fs returns 1.84 eV, larger by $(\tau/\mathrm{fs})^2 = 103.64$.

The coordinates are printed in the shortest form that reads back as the same binary64 value [27, 28], here 15 to 17 significant digits, and the reader rounds each decimal to the nearest binary64 value [29, 55]. A writer that prints six decimals instead puts $x = 1.386494$ in line 12, a shift of 1.08×10^{-7} Å, and moves the 27 coordinates by up to 4.87×10^{-7} Å. Resumed from six decimals, the climbing-image band of Section 7 converges in the same step count with its barrier moved by 1.19×10^{-8} eV. The frame ends after line 32, and the next line, if any, opens the next frame. Listing S1 of the supplementary material runs the example program of the release on this file and on the two variants of the angle line.

3. The grammar of CON 3.1

Section S1 of the supplementary material is the normative specification, 125 numbered clauses, and this section states the rules that Section 2 used, with the clauses behind them in Table 2. A CON file is one or more frames with no marker between them, followed only by lines of white space. With T

atom types of N_k atoms each, a frame $\mathcal{F}$ is

$$\begin{aligned}
\mathcal{F} &\to \lambda_1 \;\; \lambda_2 \;\; (a\ b\ c) \;\; (\alpha\ \beta\ \gamma) \;\; \lambda_5 \;\; \lambda_6 \;\; T \;\; (N_1 \cdots N_T) \\
&\quad\; (M_1 \cdots M_T) \;\; \mathcal{B}_1 \cdots \mathcal{B}_T \;\; \mathcal{S}^*, \\
\mathcal{B}_k &\to s_k \;\; (\texttt{Coordinates of Component}\ k) \;\; \left(x\ y\ z\ m\ [\iota]\right)^{N_k}, \\
\mathcal{S} &\to \varnothing \;\; \mathcal{V}_1 \cdots \mathcal{V}_T, \\
\mathcal{V}_k &\to s_k \;\; (S\ \texttt{of Component}\ k) \;\; \left(v_1 \cdots v_w\ m\ \iota\right)^{N_k},
\end{aligned} \tag{5}$$

where each symbol or parenthesised group is one line, a superscript repeats a line, λ_1, λ_5 and λ_6 are free lines, λ_2 is the header, M_k and s_k are the mass and symbol of type k, ι is the identifier, $\varnothing$ is a blank line and S is the name of a section of width w. A type count of zero is a frame without atoms. The release includes this shape as an informative parsing expression grammar (PEG) [56], and where the PEG admits a token that a clause rejects, the clause decides.

The cell rule rests on Eq. (2). readcon-core evaluates Eq. (1) in binary64, accepts the angles when the computed radicand satisfies $\hat{\rho} > -1 \times 10^{-12}$, sets $c_z = 0$ when such a $\hat{\rho}$ is negative, and otherwise rejects the frame as a validation error, a class that the specification leaves open. The window is fixed, so it accepts as flat a cell whose exact radicand lies between -1×10^{-12} and zero, and it rejects every set of angles that misses a real cell by more.

Table 2 gives the masks that each version admits and the range of the identifier. The readcon-core writer refuses a frame that repeats an identifier, and no clause binds the identifiers of one frame to those of another.

Each number is a finite decimal that the reader rounds to the nearest binary64 value, ties to even [26, 29, 55]. The default writer prints the shortest decimal that rounds back to the same value [27, 57, 58, 28], so writing a frame and reading it back returns every coordinate unchanged, and a setting prints six fixed decimals instead. A gzip or zstd envelope wraps the same text [59, 60], and a reader recognises it by content. readcon-core decodes zstd when built with that feature and otherwise refuses the file by name.

Table 3 lists the reserved keys of the JSON line. The major is `con_spec_version`, and the minor is `con_spec_minor`, zero when absent. For a frame whose header is a JSON object, let μ be its minor, U the set of keys of its `units` object, and V true when the frame has a velocity section.

Table 2: Clauses of the specification that Sections 2 and 3 apply. Section S1 of the supplementary material gives their normative text.

Clause	Rule
CON-frame.1	After the last complete frame, lines that hold only white space are ignored.
CON-frame.7	After a complete frame with its sections, or with the legacy velocity block where that applies, the next line is line 1 of the next frame, even when it is empty.
CON-token.3	Fields inside a line are separated by spaces and tabs only, so a form feed or a no-break space belongs to the field it touches.
CON-token.4	Where the PEG admits a token that a clause rejects, the clause decides.
CON-cell.1	When the header carries `lattice_vectors`, the reader takes the cell from it and not from lines 3 and 4.
CON-cell.3	A frame without `pbc` is periodic in all three directions, and a `pbc`
CON-cell.4	that is present is three booleans, one per cell vector.
CON-cell.6	Without `lattice_vectors`, the reader builds the cell of Eq. (1) from lines 3 and 4, with the angles in degrees.
CON-validate.4	With `validate` true, the reader also rejects, among other checks, a cell length that is not positive and an angle outside the open interval from 0 to 180 degrees.
CON-mask.2	In version 1 the mask is 0 for a free atom or 1 for an atom fixed in
CON-mask.3	every component, and any other token is an illegal flag.
CON-mask.4	From version 2 the mask is an integer from 0 to 7, read by Eq. (3).
CON-mask.5	A float spelling, a sign or a value outside the set of the version is an illegal flag in every mode.
CON-id.1	The identifier is an integer from 0 through $2^{64} - 1$ that is never
CON-id.3	read through binary64, a writer emits identifiers unique within a
CON-id.4	frame, and a reader accepts repeated ones.
CON-num.1	A decimal rounds to the nearest binary64 value, and a field that
CON-num.5	spells `nan`, `inf` or `infinity` with any sign and in any case, or a decimal that rounds to an infinity, is rejected as a non-finite value in every mode.
CON-units.8 to	Versions 1, 2 and 3.0 are read in `units.length` per natural time
CON-units.10	unit whatever `units.time` names, and from version 3.1 `units.time` sets the unit of the velocities.

Table 3: Reserved metadata keys of the JSON header line.

Key	Type	Key	Type
`con_spec_version`	integer	`neb_bead`	integer
`con_spec_minor`	integer	`neb_band`	integer
`section_arity`	object	`convergence_fmax`	number
`sections`	array	`convergence_energy`	number
`validate`	bool	`converged`	bool
`generator`	string	`fmax`	number
`units`	object	`bonds`	array
`pbc`	array	`storage_dtypes`	object
`lattice_vectors`	array	`chemfiles_atom_names`	array
`energy`	number	`chemfiles_atom_types`	array
`potential`	object	`chemfiles_atom_properties`	array or object
`frame_index`	integer	`time`	number
`timestep`	number		

The reader accepts that header only when

$$\begin{aligned} & M \in \{2,3\} \ \wedge \ \mu \in \mathbb{N} \\ & \wedge \ \Big(M = 3 \Rightarrow \{\text{length}, \text{energy}\} \subseteq U\Big) \\ & \wedge \ \Big(M = 3 \wedge \mu \geq 1 \wedge V \Rightarrow \text{time} \in U\Big). \end{aligned} \tag{6}$$

For $M = 3$ it also rejects a unit whose dimension is not that of its quantity. A reader accepts a later minor of a known major, keeps an unknown key on rewrite with its digits as read, and rejects any other major as an unsupported version. H5MD splits its version into the same pair [32].

The sections that `sections` lists follow the coordinate blocks in that order, each after one blank line (Table 4). A section the reader does not know is skipped only when the frame's minor is later than the reader's and `section_arity` gives its width $w \geq 1$. Each skipped row has $w + 1$ or $w + 2$ tokens, which the reader counts without parsing, and any other unknown section rejects the frame. A frame with no `sections` key, which includes every version-1 frame, admits a legacy velocity block at its end, a blank line followed by a complete block of the shape of lines 22 through 32 of Listing 1. Any other blank line there is the first line of the next frame. The text does not define a blank line, and readcon-core takes it to be a line of Unicode white space only. After the last frame it ignores one such line followed by spaces, tabs, form feeds and line ends. Table S6 of the supplementary material gives the other cases.

Table 4: Reserved sections at minor 1. Each data row carries its values, then the mask and the identifier.

Section	Values	Rule
Velocities	$v_x\, v_y\, v_z$	Time unit from 3.1
Forces	$f_x\, f_y\, f_z$	
Energies	one value	Name `energies`
Charges	one value	
Spins	one value	
Magmoms	$m_x\, m_y\, m_z$	
Displacements	$d_x\, d_y\, d_z$	`units.length`
Spreads	$s_x\, s_y\, s_z$	`units.length`, each at least 0

From minor 1, a frame with a velocity section declares `units.time`, and the reader converts that unit to femtoseconds by Eq. (4). Frames of earlier versions are read in the natural unit whatever `units.time` names (Table 2). Producers of that era wrote natural-unit numbers under an fs label, and eOn 3.6.0 still does [2]. The reader keeps such a label without applying it, and a rewrite keeps the frame at its minor with its label, so the next reader again applies the natural unit.

Table 5 is the history of the grammar. A change to the text either decides a case that the earlier text left open, for every version, or adds a requirement under a new minor, which applies from that minor on. A later minor therefore never makes a reader reject a file that an earlier minor requires it to accept. We take changes through issues and pull requests on the repository, and each change adds the corpus files that test it.

Table 5: CON versions. A row with an unmarked date takes its version, date and change from the version history of the specification. Its release is the first readcon-core release whose specification lists that version with that date, and version 1 is the text eOn writes. A date marked (tag) is the tag date of a release that changed the text of a version without a new version number, and its row names the clauses that release changed.

Version	Date	Release	Change
1	original	eOn	eOn text. Column 5 present. Column 4 is 0 or 1. No JSON.
2	2026-03-25	readcon-core 0.6.0	JSON line 2, ids, bit masks, declared sections.
3.0	2026-06-27	readcon-core 0.14.0	`units.length` and `units.energy` MUST be present. Minor absent.
3.1	2026-10-11	readcon-core 0.17.0	Minor 1. `units.time` when velocities are present. Natural-unit rule.
3.1	2026-10-29 (tag)	readcon-core 0.17.2	Adds `CON-header.4`: Parsers MUST accept an empty line 1 and an empty line 2. Adds to Sections: "A blank line that does not begin that block is line 1 of the next frame."

Version	Date	Release	Change
3.1	2026-11-07 (tag)	readcon-core 0.18.0	Drops from Specification: “Requirement words follow RFC 2119.” Adds to Header: “Line 1 MAY be empty, and parsers MUST keep that empty line.” Adds to Header: “Line 2 MAY be empty.” Drops from Header: “Line 1 may be empty, and parsers keep that empty line.” Drops from Header: “Line 2 may be empty.” Rewords `CON-id.1`: Column 5, when present, MUST be parsed as an integer in 0 through $2^{64} - 1$, and MUST NOT be parsed through binary64. Adds to Errors: “A reader that stops at the first error MUST use the lowest line number, so threads cannot change which error is reported.” Drops from Errors: “A reader that stops at the first error uses the lowest line number, so threads cannot change which error is reported.” Adds to Version history: “Unknown sections of a newer minor MAY be skipped by `section_arity` (`D14`).” Adds to Version history: “The writer MUST NOT invent or replace `con_spec_minor` (D15).” Drops from Version history: “Unknown sections of a newer minor may be skipped by `section_arity` (`D14`).” Drops from Version history: “The writer does not invent or replace `con_spec_minor` (D15).”

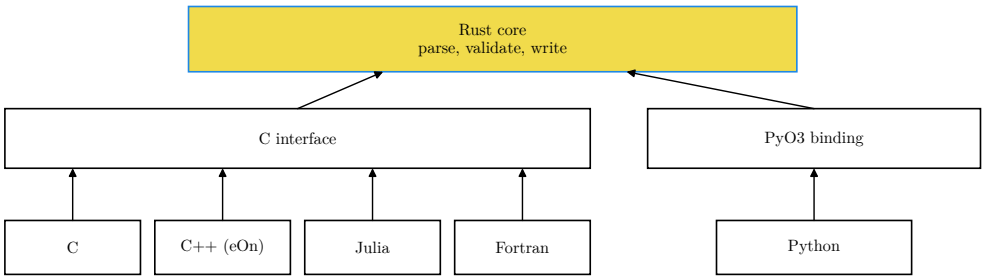


Figure 2: **One Rust core serves every language.** C, C++ (including eOn), Julia and Fortran call the core through the C interface, and Python calls it through PyO3. An arrow runs from the caller to the code it calls.

4. Reference implementation

readcon-core implements the grammar once, in Rust (Figure 2). C++, Julia [61] and Fortran 2008 call its C interface, and Python calls the Rust core through PyO3 and receives NumPy arrays [62]. eOn links readcon-core through the C++ wrapper, and its public release 3.6.0 embeds readcon-core 0.16.0, which writes CON 3.0 [2]. That layering follows chemfiles [40], one core with a binding for each language, and we take it over unchanged.

In the core, Algorithm 1 is the path reader of the release. A plain file of fewer than $B_{\mathrm{map}} = 65{,}536$ bytes is read whole into memory, a larger one is memory-mapped, and a gzip or zstd file is decompressed into memory whatever its size. One thread parses frame after frame. At the default thread count, the global thread pool parses a text of at least $B_{\mathrm{par}} = 2$ bytes, and an explicit count above one parses in a pool of that size. The parallel path walks the frame boundaries in one thread, because the start of each frame depends on the counts before it, then parses the frames independently and returns them in file order. Table S7 of the supplementary material gives the source lines of each step.

One frame of N atoms costs $\Theta(N)$ time and memory. For a text of B bytes, after inflation for a gzip or zstd file, the read costs one serial $\Theta(B)$ boundary walk and a $\Theta(B)$ parse that P threads share, and the frames it returns occupy $\Theta(B)$ memory. A compressed file of C bytes is first inflated by one serial decoder in $\Theta(C + B)$ time. The peak resident set of each reader on this branch is in Section 9.

Every writer that takes a path writes its frames to a temporary file in the

Algorithm 1 Read a CON file from a path, as readcon-core 0.18.1 does.

Require: path; thread count P, where absent or 0 selects the default
Ensure: frames in file order, or an error that names its line
Read the first four bytes; on gzip magic, decompress every member into memory
On zstd magic, decompress when built with zstd, else **error**: unsupported zstd input
Otherwise read a file shorter than B_{map} whole, else memory-map it
if $P = 1$, or no parallel feature, or P is the default and the text is shorter than B_{par} **then**
    Parse frame after frame with ParseFrame
else
    Walk the frame boundaries in one thread, skipping each frame by its counts
    Parse the frames with ParseFrame in a pool of P threads, or the global pool
end if
function ParseFrame
    error on a byte-order mark at the start of the file; line 2 is JSON when it starts with `{`, else version 1
    Check the header against Eq. (6); keep unknown keys
    Read the cell; **error** when $\hat{\rho} \leq -1 \times 10^{-12}$ in Eq. (1)
    Read the type counts, the masses and the coordinate rows
    Round each decimal to the nearest binary64 value; **error** on a value that is not finite; check each mask against Eq. (3)
    if line 2 has no `sections` key **then**
        Read a legacy velocity block when a blank line and a complete block follow
    else
        for each name in `sections`, in order **do**
            Read a reserved section, or count the rows of a later minor's section of declared width, else **error**
        end for
    end if
    error on a velocity section without `units.time` when $M = 3$ and $\mu \geq 1$
end function

directory of the destination, flushes and syncs that file once the last frame is accepted, and renames it over the destination. A frame that fails deletes the temporary file, so the destination keeps its earlier bytes. The uncompressed writers of the C interface write a device or a named pipe in place, and a writer to a string or to a stream that the caller opened writes no temporary file. Each language binding has a test that fails a write midway and compares the destination with its earlier bytes. Listing 2 reads a band from Python and writes it back to the same path, after which the file contains either its earlier frames or the frames written.

Listing 2: **A checkpoint round trip in Python.** The default printer keeps every binary64 value; six decimals are a separate argument.

```
import readcon
frames = readcon.read_con("band.con")
readcon.write_con("band.con", frames)
```

5. Zero-copy tensor exchange

A caller that already has a frame should read its coordinates as a tensor. readcon-core exposes that tensor through DLPack [63]. On the Rust builder the methods for positions, velocities, forces, per-atom energies and masses return a borrowed float64 tensor, and the method for atom identifiers returns a borrowed uint64 tensor. The lifetime of each borrowed tensor is the builder, and the default build places it on the CPU. A C caller that needs the tensor after the builder is gone takes the versioned managed tensor, and that export copies the storage. Python materialises a contiguous NumPy array, and that array implements the DLPack protocol, so PyTorch, JAX and CuPy take the array with no second copy. The measured exchange is a NumPy DLPack view against an explicit copy of the same positions, for 32768 copper atoms. The log records eight repeats of each arm. The view takes a median of 4.799 microseconds and the copy a median of 60.69 microseconds, so the view costs 0.07908 times the copy. A 95 percent bootstrap interval for that ratio, drawn from those eight samples, is 0.06108 to 0.09779, and the interval excludes 1. The type 7 interquartile ranges are 1.338 microseconds and 15.29 microseconds. Figure 3 shows the eight repeats. The log is `paper/cpc/logs/dlpack-exchange.log`.

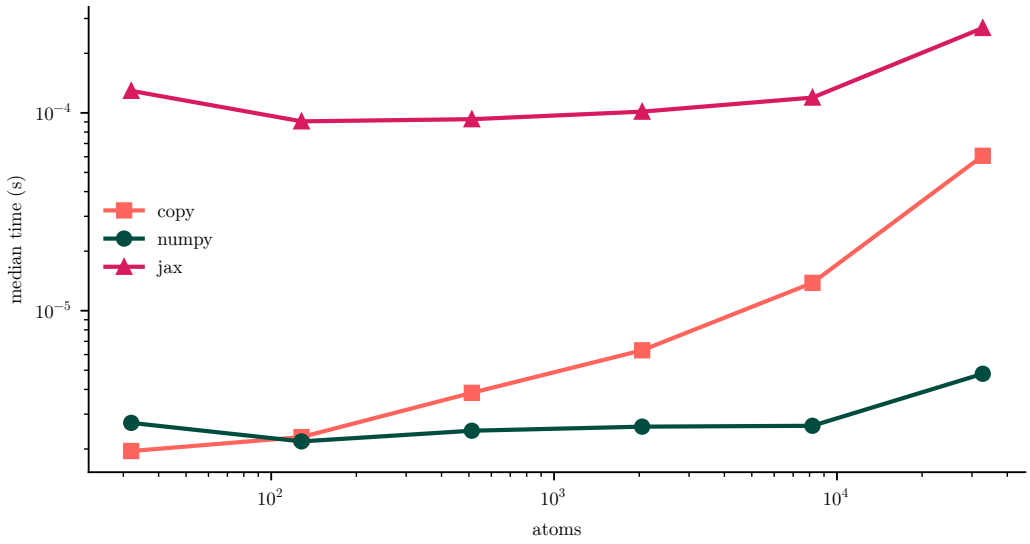


Figure 3: **A borrowed DLPack view costs 0.07908 times a copy of the same positions.** Eight repeats on 32768 atoms. The bar is the median and the whisker is the type 7 interquartile range.

6. Conformance

A specification settles a dispute between readers only when each reader can be tested against it. The conformance corpus of readcon-core 0.18.1 contains 14 legal and 8 illegal files (Table S1), and its manifest names the error class of each illegal file. Read through the Python module of the release, all 22 files return the verdict and error class of their manifest entries, and the release checks the same files from Rust, Python, C, Fortran and Julia.

Table S1 is in the supplement.

That corpus has few files for the header, the sections and the units, so we wrote a second corpus of 143 CON 3.1 files, 62 legal and 81 illegal. Its manifest names for each file the clauses that decide it and the outcome, the frame count and periodic flags of a legal file or the error class and line of an illegal one. For 95 entries the outcome follows from the text of a clause, and the others record a reading where the text leaves a case open (Section S4). All 143 files return the outcome of their entry at one, two and eight threads and at the default. Both corpora also go through every read path of the Python module and the C library, in the default and the parallel build at each of those thread counts, and all $9,393$ reads return the outcome that the

manifests name (Table S9).

We also wrote an independent reader in 884 Python source lines that import nothing from readcon. It follows the clauses of the specification, and where a clause requires a rejection without naming its class, or no clause decides a case, it follows the reading of the corpus manifest. Of the 165 files of both corpora, the frame list and the frame iterator of readcon and the independent reader each return the manifest outcome on all 165. On the 76 files that both readers accept, all 76 carry the same bits in every coordinate, velocity and cell entry and equal masks, identifiers, units and periodic flags (Table S10). The two readers also read 21,000 mutants of the 132 corpus files under 4,000 bytes, 3,000 from each of 7 seeds, each made by inserting, deleting or replacing characters and tokens at one to three random places. They agree on 20,985 of them, with the same frame count or error class. All 15 differing cases fall in the count-line group. The record leaves 0 cases unexplained.

The release runs 140 differential cases mutated from its test files and its conformance corpus. Every case is read from a string, from bytes, from a plain and a gzip file at one to sixteen threads, through the column readers and through the stream reader, and each read returns the frame count, coordinates and section presence of the one-thread string read or stops with its error at the same line. Of the 125 clauses of the specification, 121 are named by a corpus file, a probe or a test of the library, and the other 4 state no duty of a reader, such as the rules for revising the specification.

One name must mean one language for every reader that carries it. The published wheels of readcon-core 0.17.0, 0.17.1, 0.17.2 and 0.17.3 do not return the same outcome on every probe file. On files of two frames whose first line is empty, the oldest refuses the file and the later three read two frames, and on a blank symbol line the newest two refuse a file that the earlier two read (Section S10). Each of those changes decided a case that the text of its release left open, and Table 5 lists the clauses involved.

Extended XYZ is the text format to set beside CON on this test, because it has a published grammar, the specification of the libAtoms extxyz package [30], and parsers in C, C++ and Python. Table S2 gives seven three-atom files to 7 reader paths from 6 libraries. On the reference frame the 5 extended XYZ paths, from 4 libraries, return 3 different outcomes. That grammar admits a `d` or `D` exponent and reads `T`, `true`, `True` and `TRUE` as true, and the paths split on both files. It leaves open whether a comment line that is not key-value pairs makes a file plain XYZ or an error, and on such a file the C parser of libAtoms departs from the default periodicity while its Python

parser fails.

Table S2 is in the supplement.

7. Resumed saddle searches

The reads in Section 9 use the library built on this branch. We first wrote the survey frame of Listing 1 with one library and read it back with the same library or another, 15 routes in all, 10 within one library and 5 across two (Table S3). The libraries are ASE [23], chemfiles [40], MDAnalysis [42, 64], h5py, OVITO [44] and Open Babel [45], at the versions of Section S10, and each stores each field through the mechanism its documentation gives. We grade a returned field as fixed by the format when the format's own specification defines its place in the file, and as a convention when only the documented layout of a library defines that place.

Table S3 is in the supplement.

Only the CON 3.1 route returns every field from places that its own specification defines. The ASE trajectory, with the identifiers stored as ASE tags, and H5MD written and read with h5py also return every field bit for bit. The trajectory keeps the mask, the identifiers, the unit and the periodicity in the layout of ASE, and H5MD keeps the mask in a dataset outside the H5MD specification, which only our reader knows to read. Extended XYZ returns the mask and the identifiers through the column names of ASE, which chemfiles and OVITO also read. ASE writes that file with eight decimals, a shift of 4.7×10^{-9} Å; chemfiles moves the coordinates by 4.9×10^{-6} Å, rounds the cell lengths to six significant digits and drops the velocities. MDAnalysis returns single precision, which changes a coordinate by up to 2.2×10^{-7} Å, and Open Babel keeps the identifiers only inside the site labels of a Crystallographic Information File (CIF). Identifiers come back on 4 of the 5 cross-library round trips, all through the `id` column of ASE.

7.1. A climbing-image band through every route

The restart study uses a larger slab, Pt(100) under the effective-medium theory (EMT) potential of ASE [65], three layers of 3×3 atoms with the bottom layer held in every component and one adatom held in z at the height of Table 6 (Figure 1, right). The adatom hops between neighbouring fourfold hollows along x. With the slab fixed, the EMT force on the adatom at that height exceeds the band tolerance. That height is no stationary point of the free adsorbate, so a file that drops the hold resumes another relaxation. A

Table 6: Parameters held fixed for the climbing-image case study.

Symbol	Meaning	Configuration key	Value
L_{vac}	vacuum on each side of the slab	slab `vacuum`	6.0 Å
z_{ad}	adatom height above the top layer	adsorbate height	1.75 Å
f_{end}	endpoint force tolerance	minimise `fmax`	0.002 eV/Å
f_{band}	band force tolerance	band `fmax`	0.01 eV/Å
N_{img}	movable images	band images	5
k_{sp}	spring constant	band `k`	0.1 eV/Å^2
Δs	steepest-descent step	band step size	0.02 Å^2/eV
K	interrupt step	band interrupt	25
N_{cap}	iteration cap	band step limit	4000

climbing-image band of five movable images with the improved tangent [9, 10] converges under fixed-step steepest descent on the projected force, which keeps no step history, so a difference after the restart comes from the file. The spring and the tangent are the defaults of ASE, the band tolerance is the default force tolerance of the ASE optimiser divided by the five movable images, and each endpoint is minimised to that tolerance divided by five again [23]. A force equal to the band tolerance moves an atom by 2×10^{-4} Å under that step. Uninterrupted, the band converges in 60 steps and 61 force calls, with barrier 0.6446 eV and 56 free modes on the middle image. We interrupt it after K steps, write it, read it back and resume it to f_{band}, once per route of Table 7.

The shortest CON round trip changes no coordinate, and the resumed band is the uninterrupted band bit for bit: the same 60 steps, 56 free modes on the middle image, and the same coordinates and energy on every image (Figure 4). Every force evaluation builds its EMT calculators afresh, so a step depends on the positions alone. The ASE trajectory with identifiers in tags and H5MD written and read with h5py return the same row. POSCAR with the height held as FixScaled moves one coordinate by 5×10^{-17} Å and shifts the resumed barrier by -2.0×10^{-14} to 3.7×10^{-14} eV. Six decimals move the file by 4.95×10^{-7} Å and the barrier by 1.19×10^{-8} eV, and extended XYZ written by ASE moves them by 4.99×10^{-9} Å and 1.09×10^{-9} eV, all in the same step count. Each shift is printed to the figures on which reruns on the linear-algebra builds of four host generations agree, or as their range (Section S7).

Table 7: Resumed climbing-image bands of the EMT Pt(100) adatom hop of Table 6, one row per route of the round trip: stopped after 25 steepest-descent steps, written and read back, then resumed to 0.01 eV/Å. File is the largest coordinate change on the round trip, and modes are the free Cartesian degrees of freedom of the middle image. Before the write every image carries atom identifiers that are not the row order, each route in its own carrier for them, and kept lists which of the per-component mask, those identifiers, the periodic flags (pbc) and the format version the read returns on every image. Barrier is the highest image energy minus the first, and $\Delta E^{\ddagger}$ and the displacement are against the uninterrupted band. Force calls count the resumed leg, or the whole band in the uninterrupted row. Wall is the median wall-clock time of that leg alone over 11 interleaved fresh processes, with the interquartile range of those times beside it, so building the band, the first 25 steps and the round trip are outside the clock. POSCAR (ASE), FixScaled z: adatom written T T F. H5MD (h5py), private mask dataset: fixed is not an H5MD element, so only a reader that knows this layout gets the mask back. [a]29 units in the last place of a 0.009423 Å coordinate. [b]21 units in the last place of the 8.771 eV highest-image energy. [c]3 units in the last place of a 9.855 Å coordinate. Section S7 gives the linear-algebra builds on which the printed figures agree.

Route	File (Å)	Modes	Kept	Steps	Force calls	Wall (s)	IQR (s)	Barrier (eV)	$\Delta E^{\ddagger}$ (eV)	Max displacement (Å)
uninterrupted	–	56	–	60	61	1.608	0.006	0.6446	0	0
CON 3.1, shortest round trip	0	56	mask, ids, pbc, version	60	36	0.955	0.007	0.6446	0	0
CON 3.1, six decimals	4.95×10^{-7}	56	mask, ids, pbc, version	60	36	0.956	0.011	0.6446	1.19×10^{-8}	5.69×10^{-7}
extended XYZ (ASE)	4.99×10^{-9}	56	mask, ids, pbc	60	36	1.008	0.006	0.6446	1.09×10^{-9}	4.94×10^{-9}
ASE trajectory, ids in tags	0	56	mask, ids, pbc, version	60	36	0.951	0.006	0.6446	0	0
POSCAR (ASE), FixScaled z	5×10^{-17}[a]	56	mask	60	36	0.965	0.007	0.6446	-2.0×10^{-14} to 3.7×10^{-14}[b]	2.7×10^{-15} to 5.3×10^{-15}[c]
H5MD (h5py), private mask dataset	0	56	mask, ids, pbc, version	60	36	0.953	0.005	0.6446	0	0

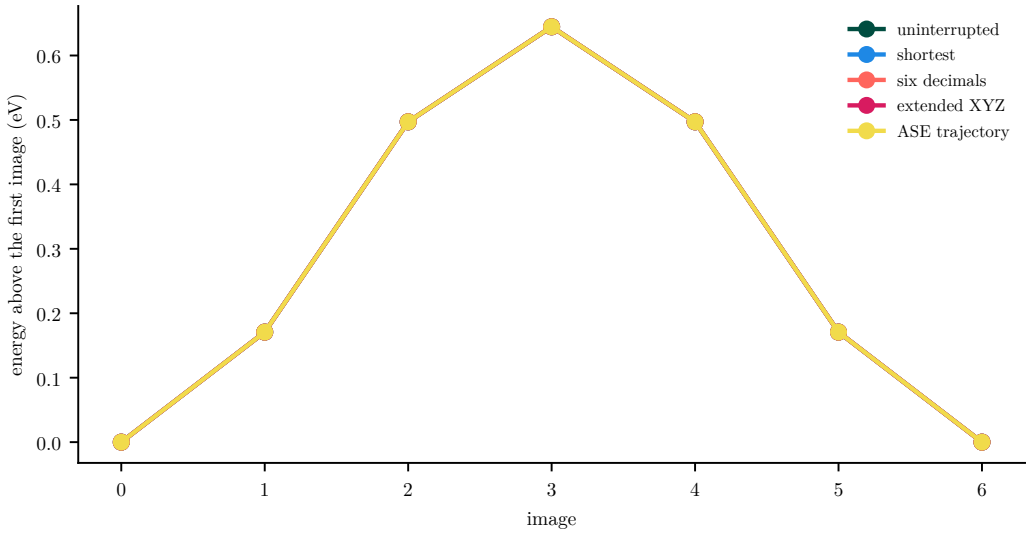


Figure 4: **The CON 3.1 restart lies on the uninterrupted band.** Energy of each image above the first image, from the stored EMT bands. The uninterrupted band and the shortest CON 3.1 round trip overlie each other. Extended XYZ, six decimals and the ASE trajectory are drawn together.

Table 8: The FIRE climbing-image band of the Table 7 slab after each reading of its endpoints. The endpoints are written and read back before they are relaxed, then the band and the Hessians use the constraints that reader returns. Modes and imaginary count the free coordinates of the highest interior image. Before the Hessians are taken, Newton steps on the free coordinates bring the relaxed endpoints to their minima and, where it has one imaginary mode, that image to its saddle. Barrier to four significant figures, prefactor to three.

Reading	Adatom free	Modes	Imaginary	Barrier (eV)	Vineyard prefactor (Hz)
height held (z fixed)	x y	56	1	0.6446	9.86×10^{10}
height not held	x y z	57	1	0.6815	3.63×10^{12}
CON 3.1, shortest round trip	x y	56	1	0.6446	9.86×10^{10}
ASE trajectory	x y	56	1	0.6446	9.86×10^{10}
POSCAR (ASE), FixScaled z	x y	56	1	0.6446	9.86×10^{10}
H5MD (h5py), private mask dataset	x y	56	1	0.6446	9.86×10^{10}

7.2. The mask and the prefactor

A barrier is one number, and the harmonic prefactor needs the free modes at the saddle. In harmonic transition state theory the rate is $k = \nu \exp(-\Delta E^{\ddagger}/k_{\mathrm{B}}T)$ with the Vineyard prefactor [6]

$$\nu = \frac{\prod_{i=1}^{n} \nu_i^{\mathrm{min}}}{\prod_{i=1}^{n-1} \nu_i^{\ddagger}}, \tag{7}$$

where the products run over the n free modes of the minimum and the $n-1$ real modes of the saddle. A mask that frees one more component can move the saddle itself, besides adding a mode to each product. We wrote the endpoints of the same slab through each route and read them back before relaxing them, converged a climbing-image band with the fast inertial relaxation engine (FIRE) [66], and took the Hessian of the highest image. With the height held, the barrier is 0.6446 eV on a path of 2.927 Å. The highest image has 56 free modes, one imaginary mode of -5.40 meV with 0.980 of its weight on the lateral motion of the adatom, and a prefactor of 9.86×10^{10} Hz. CON 3.1, the ASE trajectory, FixScaled POSCAR and h5py H5MD all return that prefactor (Table 8). With the height free, the problem a reader solves when the file has lost the hold, the band crosses a saddle at 0.6815 eV with 57 free modes and a prefactor of 3.63×10^{12} Hz, 37 times the held value.

7.3. eOn resuming a Morse band

The public eOn client resumes from its own output files. Section 9 times `eon.fileio.loadcons` on every frame at 1 thread. It times `Matter.con2matter` on the first frame only, and that row is not a ratio

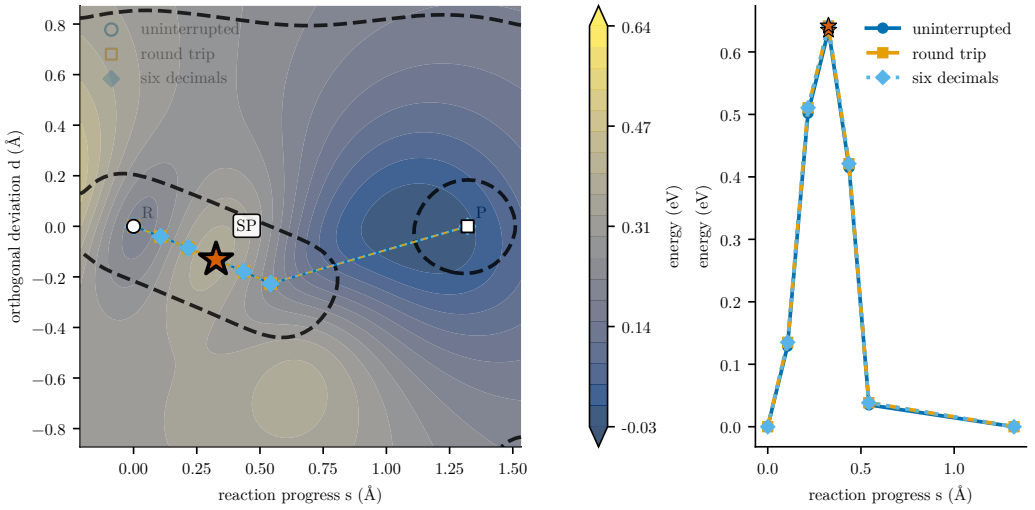


Figure 5: **The resumed Morse band ends on the uninterrupted barrier.** Left: force calls of the uninterrupted steepest-descent band and of the three resumes. Right: the barrier of each route. The horizontal line is the uninterrupted barrier. The four routes use teal, sky, coral and magenta, and the reference line is sunshine.

against a full read. The C++ row calls `readcon::read_all_frames` and then `con2matter` on every frame that call returns. ASE ase.io (format eon) is the independent CON reader timed beside `loadcons`, both on every frame at 1 thread. The default band optimiser of eOn is the limited-memory form, L-BFGS, of the Broyden, Fletcher, Goldfarb and Shanno update [67, 68], and its memory vectors are stored in no geometry file. A restart therefore cannot repeat an L-BFGS path from the geometry alone.

7.4. The velocity unit

A 28-atom fixture stores velocities in the declared natural unit. Its kinetic energy is 92.02 eV in that unit and 9,537 eV with the same numbers read as Å/fs, the factor 103.64 of Eq. (4). After one velocity Verlet step of 1 fs [69], the misread positions differ from the declared ones by 0.096 Å root mean square and by at most 0.315 Å (Section S8). A 10,001-atom slab drawn at 300 K has a kinetic energy of 390.1 eV, and the misread gives 4.043×10^4 eV.

8. Read performance

The comparison measured on this branch is Section 9. It reads one 100000-frame file, six times, one fresh process per sample.

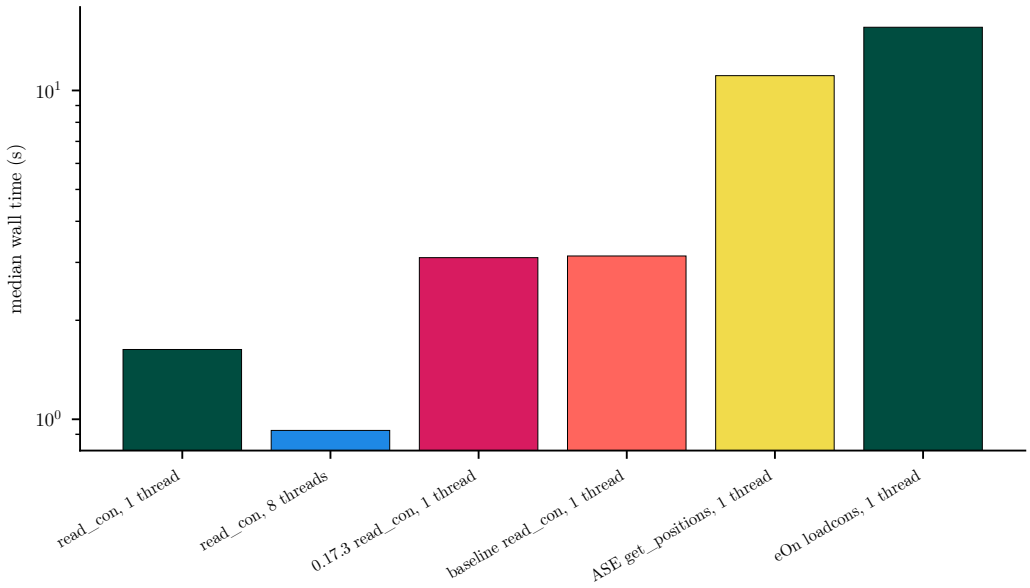


Figure 6: Wall time of the full-frame CON readers on one 100000-frame file, six interleaved repeats. The bar is the type 7 median. `read_con` is shown at 1 thread and at 8 threads. The other bars are 1 thread. The log is `paper/cpc/logs/reader-exchange.log`.

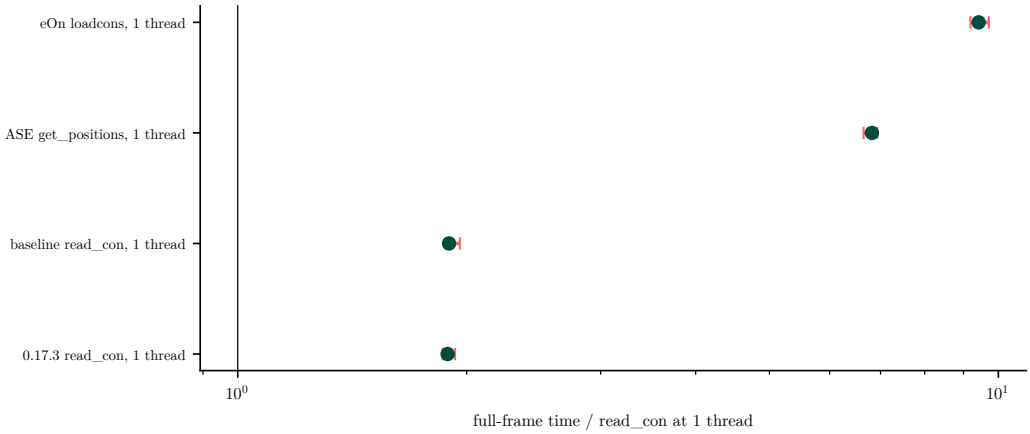


Figure 7: Full-frame readers at 1 thread, each divided by `read_con` at 1 thread, on one 100000-frame file. Each point is the median of the six per-repeat ratios, and the whisker is their type 7 interquartile range. The text quotes the ratio of the medians. The horizontal line is 1. A point above the line means `read_con` at 1 thread is that many times as fast. `read_positions` is not on this figure.

Table 9: Bytes per atom of the payloads of Section 8, raw and after gzip at level 6. Every file stores, for every frame, the species, the positions, the per-component fixed mask, the identifiers, the cell and the periodic flags. The timed trajectory also stores the velocities. No file stores another per-atom field. The first frame is frame 0 of the timed trajectory without velocities. The timed trajectory is the file the campaign reads, all 10,000 frames with velocities, per atom row. CON 3.1 is the default writer of readcon-core 0.18.1. Extended XYZ prints the shortest decimal that returns each binary64 value, H5MD 1.1 written by h5py stores every real value as binary64, and the ASE trajectory comes from the ASE writer, with the mask as a FixAtoms constraint and the identifiers as tags.

	First frame		Timed trajectory	
Format	raw	gzip	raw	gzip
CON 3.1	50.74	19.44	94.54	30.84
extended XYZ	52.70	18.89	90.66	29.54
ASE trajectory	44.42	18.88	68.61	24.14
H5MD	112.44	22.47	48.41	23.50

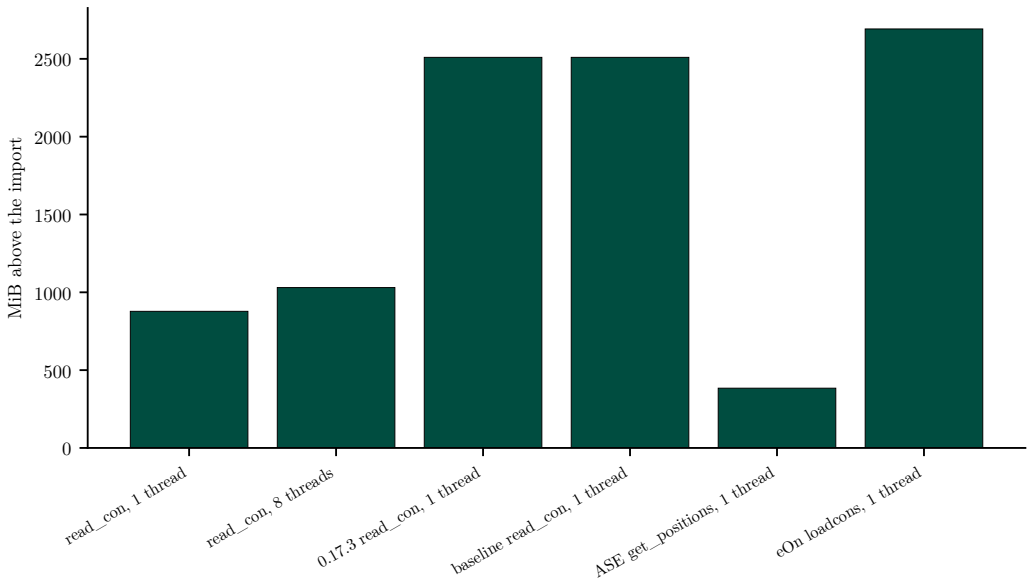


Figure 8: MiB added to the high-water mark by the read, after the import, six repeats.

9. Measurements on this host

We read one trajectory of 100000 frames and 32 copper atoms, six times for each arm, interleaved, each sample a fresh process. Each library is imported before the clock starts. The host is an Intel Xeon Processor, CPU family 6 model 207, at 2.4 GHz, with 8 cores and 47 GiB of memory. It runs kernel 6.12.113+ on Ubuntu 24.04.5 LTS. rustc 1.88.0 builds the library, and g++ 13.3.0 builds the eOn C++ arm. The trajectory is on an ext4 filesystem. The page cache is warm, because the files are written once before the samples. The log names measurement commit `e286bf570120069cbdf221d14219a381dd3eda2a`. The release arm is readcon-core 0.17.3, commit `12ca800bc2212c565941e04094cba6b4d99a62e0`, `read_con` at 1 thread. The baseline arm is readcon-core 0.17.3, commit `ac4f437989dea5aad9f75ff045af09ccf7127eba`, `read_con` at 1 thread. CON readers read the CON file, XYZ readers read the XYZ file, and HDF5 readers read the same HDF5 file. readcon reads neither XYZ nor HDF5. Memory is the VmHWM after the read minus the VmHWM after the import. `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS`, `NUMEXPR_NUM_THREADS` and `RAYON_NUM_THREADS` are set to the thread count of the arm.

9.1. Results

On the coordinate file, `read_con` at 1 thread takes 1.522 s. The same call at 8 threads takes 0.876 s, which is 1.74 times as fast as the 1-thread call. readcon-core 0.17.3 `read_con` at 1 thread takes 2.977 s. At 1 thread, `read_con` on the measurement checkout is 1.96 times as fast as readcon-core 0.17.3. ASE ase.io (format eon) calls `get_positions` at 1 thread and takes 11.16 s. `read_con` at 1 thread is 7.33 times as fast as that call. eOn 3.6.0 `loadcons` reads every frame at 1 thread and takes 14.61 s. `read_con` at 1 thread is 9.6 times as fast as `loadcons`. `read_positions` returns one coordinate array and builds no per-frame object. At 1 thread it takes 0.7893 s, and at 8 threads it takes 0.1769 s. Those are absolute times. They are not ratios against ASE or `loadcons`, which build a frame object per record. The C++ row calls `read_all_frames` and then `con2matter` on every frame. It takes 1.514 s. readcon-db ingests the same file in 11.53 s. That row writes a corpus and then reads the corpus, so it is not a speedup against a read of the file. Every all-frame arm matches 100000 of 100000 frames, and the failure count is 0. `read_con` at 1 thread adds 878.7 MiB. ASE ase.io (format eon)

adds 383.9 MiB. chemfiles adds 166.7 MiB and OVITO adds 172.3 MiB, and those two figures read XYZ at 1 thread.

The first-frame rows read one frame of the same file. They are not compared with a full read. `Matter.con2matter` takes 0.3058 s and matches 1 of 1 frame. `read_first_frame` takes 0.03137 s and matches 1 of 1 frame. The eOn build log is `paper/cpc/logs/eon-build.log`, and it names eOn commit `579738a9387e8d0ea55496f45dd057812cfab892`.

9.2. Same-file and nearest-native panel

The same-file comparison reads one con file. The readers are readcon from C, Fortran, and Python, the upstream eon client, eon Python, and the Atomic Simulation Environment (ASE) with the eon format. The nearest-native panel reads different files that carry the same atoms and the same cell. NumPy reads an npy coordinate array, which has no cell. ASE and chemfiles read extended xyz. OVITO reads extended xyz. pymatgen reads a POSCAR. MDAnalysis reads a protein data bank file with a crystal record. That panel is separate from the con table. A row passes only when the atom count, the cell, and the coordinates match. NumPy has no cell, so that row fails the cell check. The times for XYZ, HDF5 and the mapped binary coordinate file are context for choosing a format, in Section S9. h5py reads the HDF5 dataset at 1 thread in 0.5963 s. `read_con` at 1 thread reads the coordinate file in 1.522 s. The mapped binary file reads them in 0.007035 s.

We ingest 40 frames of 8 copper atoms into readcon-db and into the ASE database, six times each. readcon-db takes a median of 0.004672 s and the ASE database takes 0.07232 s. Both stores keep all 40 frames. The log is `paper/cpc/logs/db.log`.

We time a distributed read of 4000 frames of 64 copper atoms at 1, 2, 4 and 8 ranks, in shared mode and in scatter mode, 6 launches at each count. In shared mode the median is 0.054082 s at 1 rank, 0.027438 s at 2 ranks, 0.015909 s at 4 ranks and 0.0099308 s at 8 ranks. In scatter mode the median is 0.053252 s at 1 rank, 0.030186 s at 2 ranks, 0.016637 s at 4 ranks and 0.0098786 s at 8 ranks. Eight ranks are faster than four in both modes. The scatter-mode interquartile range at eight ranks is 0.00080269 s. Rank 0 reads the file once to build the index, and every other rank reads only its own byte range. In scatter mode the waiting ranks sleep on a file lock, which needs a filesystem with coherent flock across the ranks, and where flock is not coherent they fall back to the blocking broadcast. The log is `paper/cpc/logs/mpi.log`. The fallback log is `paper/cpc/logs/mpi-fallback.log`.

Table 10: Coordinate reads of 100000 frames, six fresh processes, type 7 median. Each row names the call and the thread count. `read_con` builds a frame per record. `read_positions` returns one coordinate array. ASE `get_positions` and eOn `loadcons` build every frame on 1 thread. The C++ row is `read_all_frames` and then `con2matter` on every frame. readcon-db is an ingest of the file followed by a read of that corpus. The last two rows read one frame. A one-frame row is not a ratio against a full read, and the ingest is not a ratio against a read.

Reader	Median (s)	IQR (s)	MiB	Matched
0.18.1 `read_con`, 1 thread	1.522	0.0165	878.7	100000/100000
0.18.1 `read_con`, 8 threads	0.876	0.0188	1032	100000/100000
0.18.1 `read_positions`, 1 thread	0.7893	0.0138	202.7	100000/100000
0.18.1 `read_positions`, 8 threads	0.1769	0.00779	202.9	100000/100000
0.17.3, tag v0.17.3 `read_con`, 1 thread	2.977	0.0981	2510	100000/100000
0.17.3, untagged `read_con`, 1 thread, baseline	2.971	0.0641	2510	100000/100000
ASE `get_positions`, 1 thread	11.16	0.0898	383.9	100000/100000
eOn `loadcons`, 1 thread	14.61	0.293	2692	100000/100000
`read_all_frames` then `con2matter`, 1 thread	1.514	0.0529	1059	100000/100000
readcon-db ingest, 1 thread	11.53	0.228	398.4	100000/100000
`read_first_frame`, 1 frame	0.03137	0.000573	127.7	1/1
eOn `con2matter`, 1 frame	0.3058	0.0156	263.1	1/1

Table 11: Distributed read of 4000 frames of 64 copper atoms. The median and the interquartile range are type 7, and every repeat is kept.

Mode	Ranks	n	Median (s)	IQR (s)
shared	1	6	0.054082	0.0036356
shared	2	6	0.027438	0.0015019
shared	4	6	0.015909	0.00071101
shared	8	6	0.0099308	0.00044478
scatter	1	6	0.053252	0.001088
scatter	2	6	0.030186	0.0012589
scatter	4	6	0.016637	0.00031017
scatter	8	6	0.0098786	0.00080269

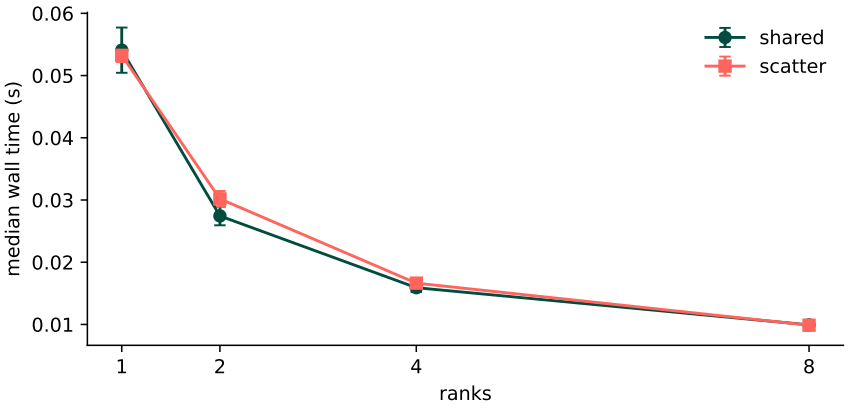


Figure 9: Distributed read on this host. The marker is the type 7 median and the whisker is the type 7 interquartile range.

10. Stored campaigns

The figures in this section are drawn by `scripts/plot_restored.py` from the records under `paper/cpc/data`. The database and MPI figures in this section are earlier stored records. Section 9 gives the repeats measured on this host.

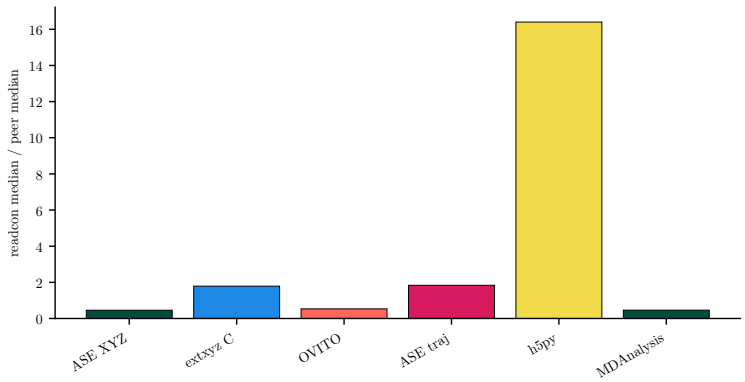


Figure 10: An earlier stored ratio of readcon-core wall time to the peer wall time.

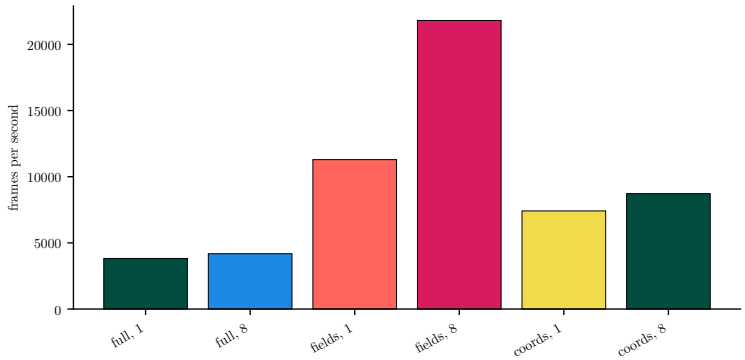


Figure 11: Frame rate of the full, field and coordinate Python reads at one and eight threads.

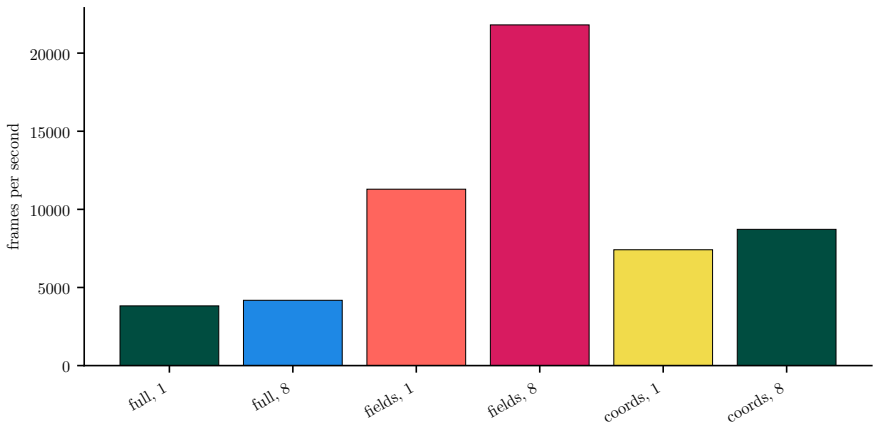


Figure 12: The same three Python reads at one and eight threads, in frames per second.

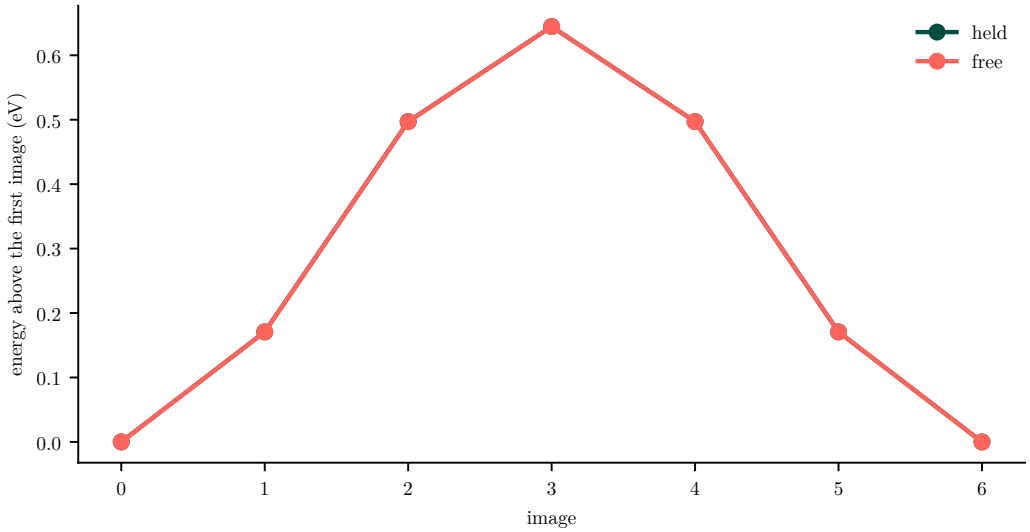


Figure 13: Image energies for the height-held POSCAR band and the uninterrupted band.

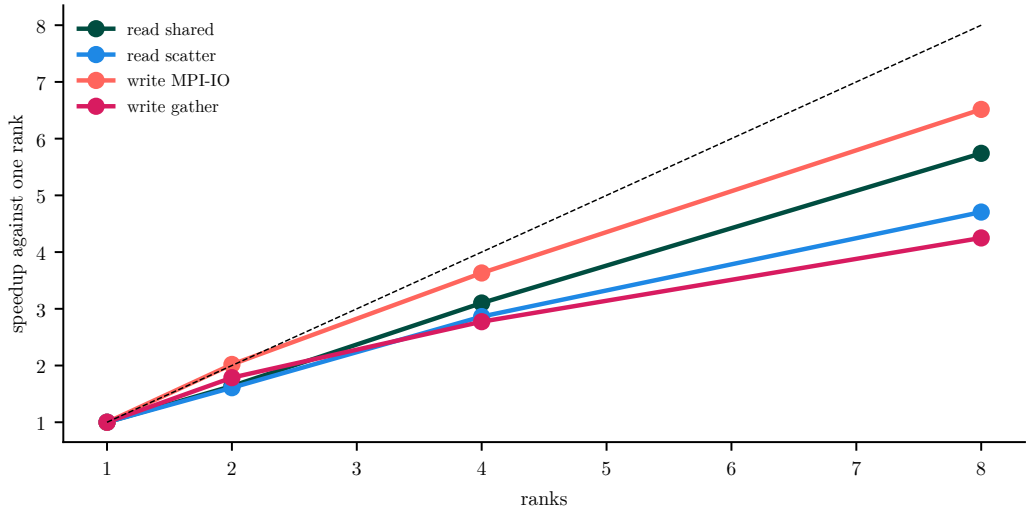


Figure 14: Earlier stored MPI speedups.

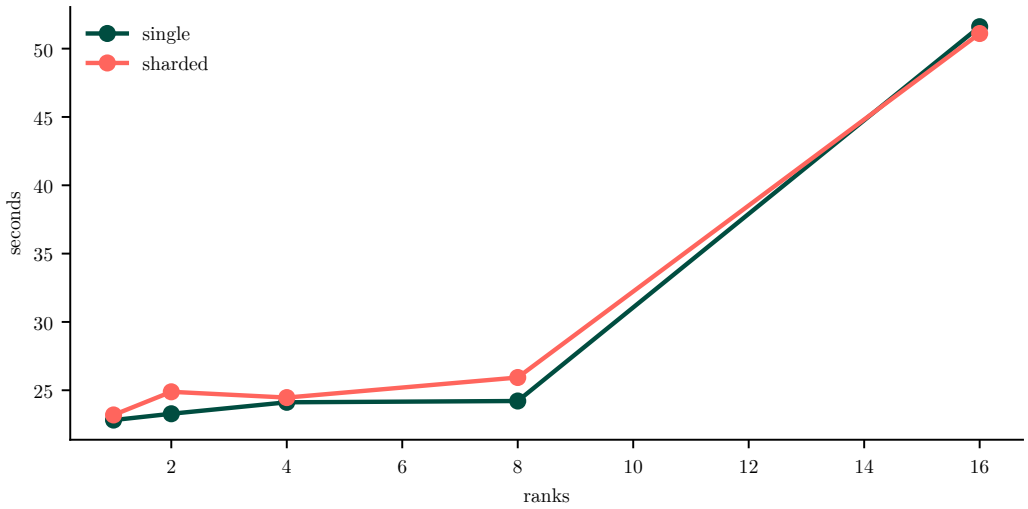


Figure 15: Earlier stored MPI ingest times.

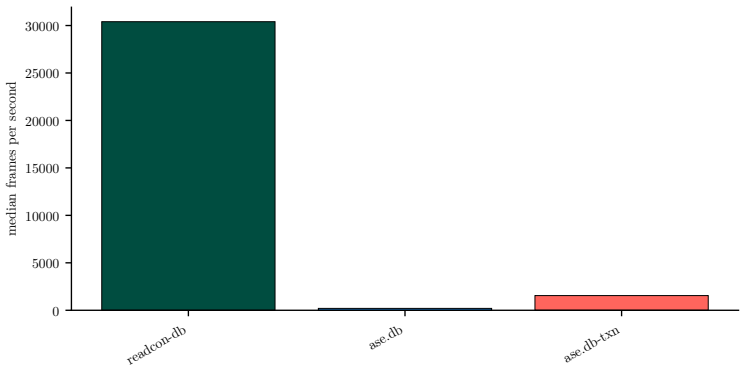


Figure 16: Earlier stored ingest rates.

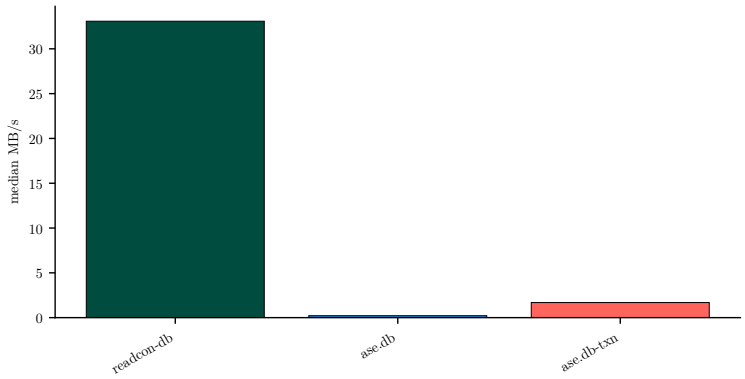


Figure 17: Earlier stored ingest bandwidth.

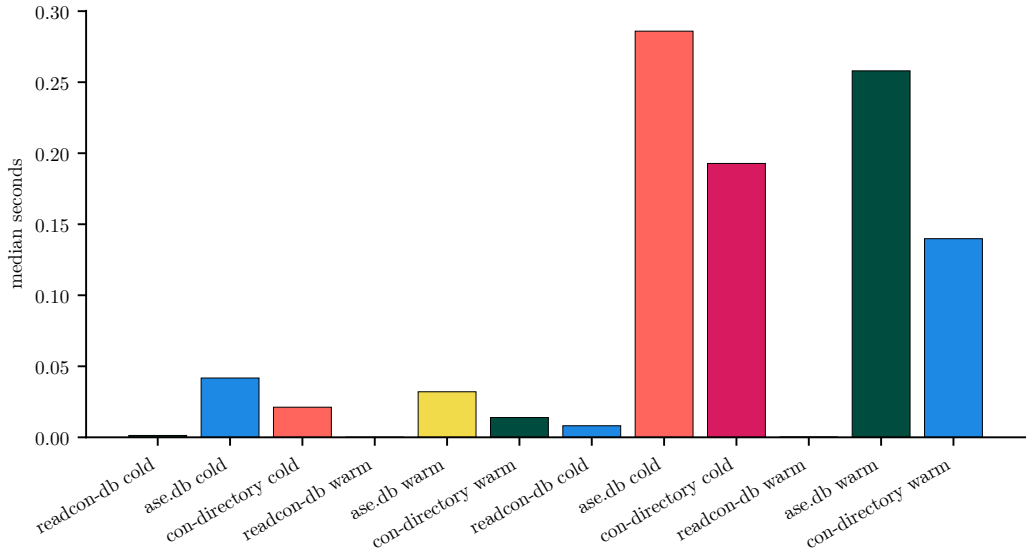


Figure 18: Earlier stored composition-query times.

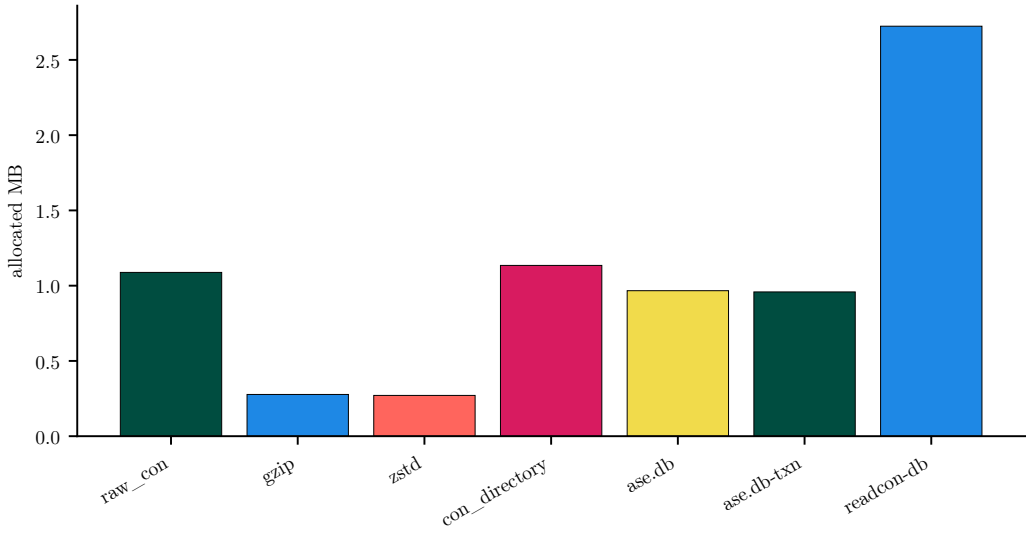


Figure 19: Allocated size of the stored 1000-frame copies.

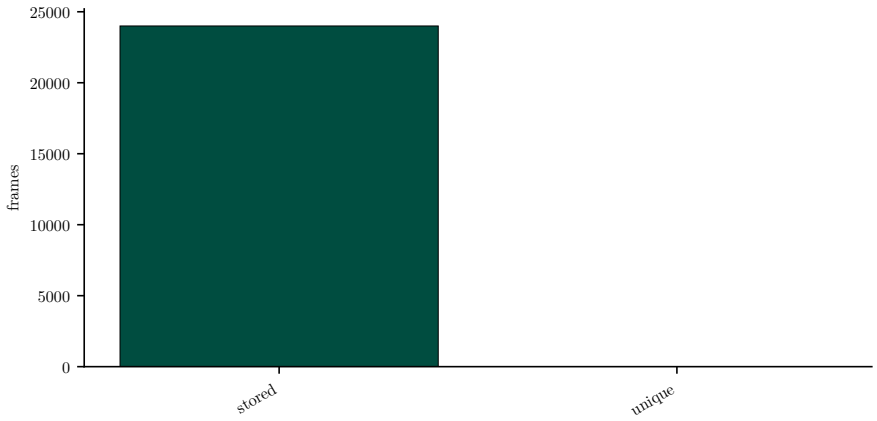


Figure 20: Stored frames against unique frame hashes in the deduplication record.

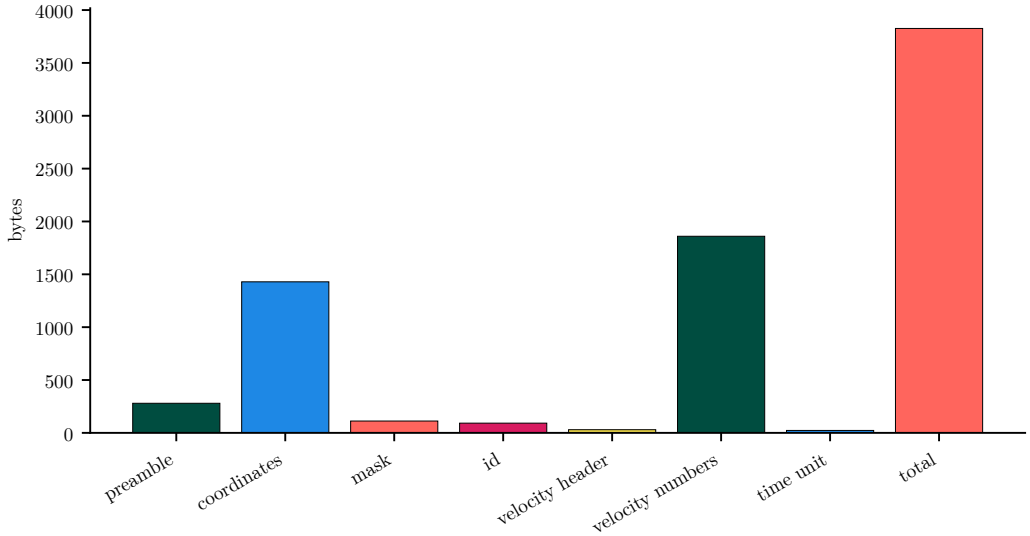


Figure 21: Byte width of each span of one stored CON frame.

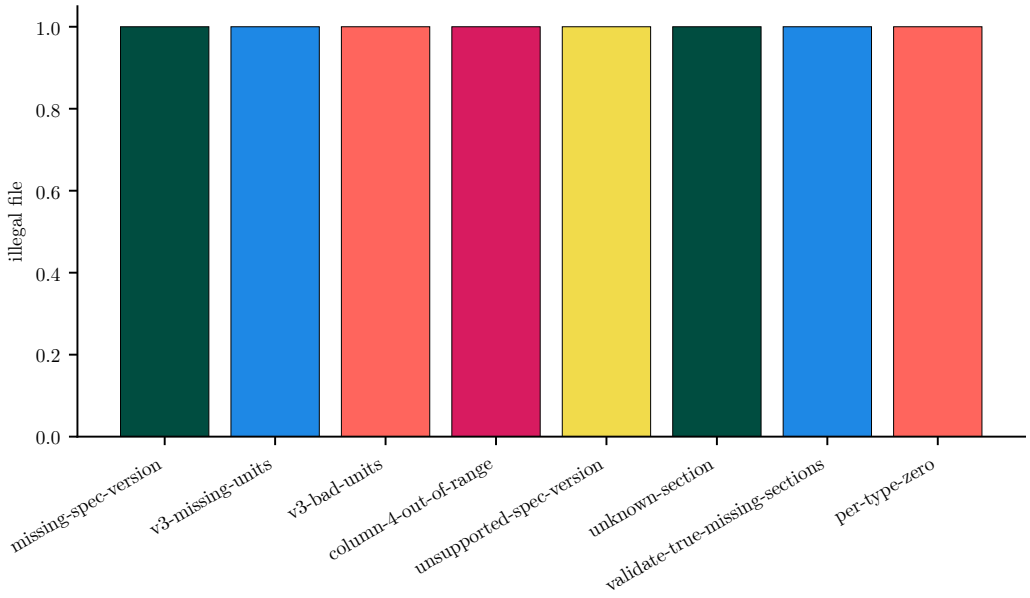


Figure 22: One bar for each illegal file in the stored error record.

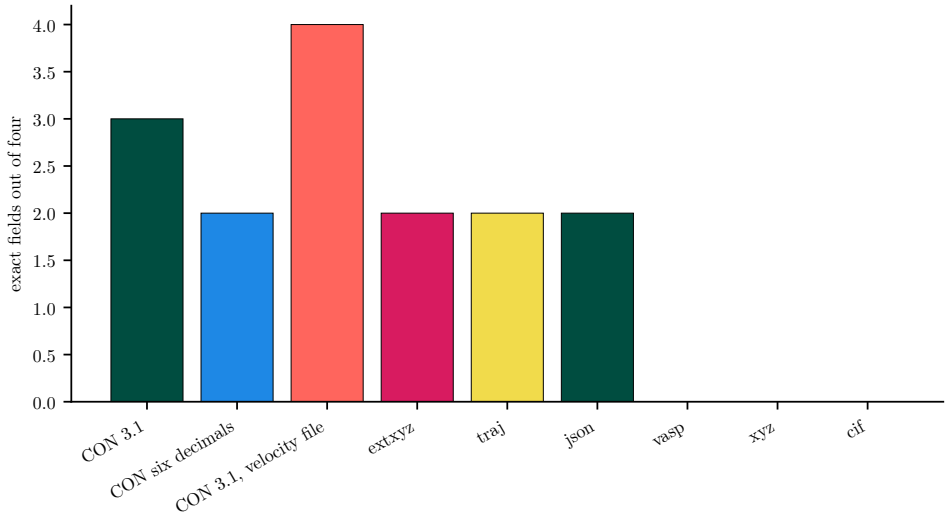


Figure 23: Count of exact mask, identifier, time unit and binary64 fields after a write and a read.

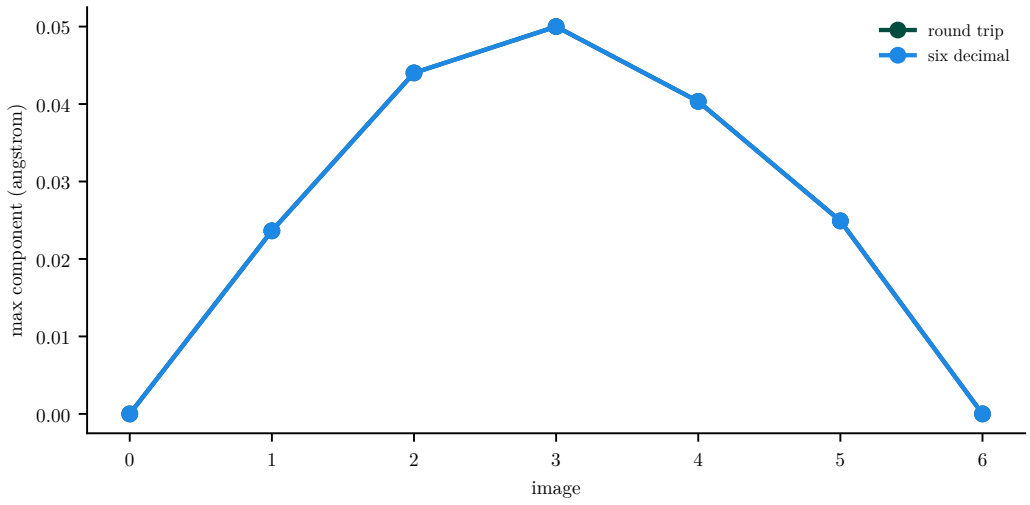


Figure 24: Largest absolute component of the minimum-image displacement, in angstrom, along each stored resume.

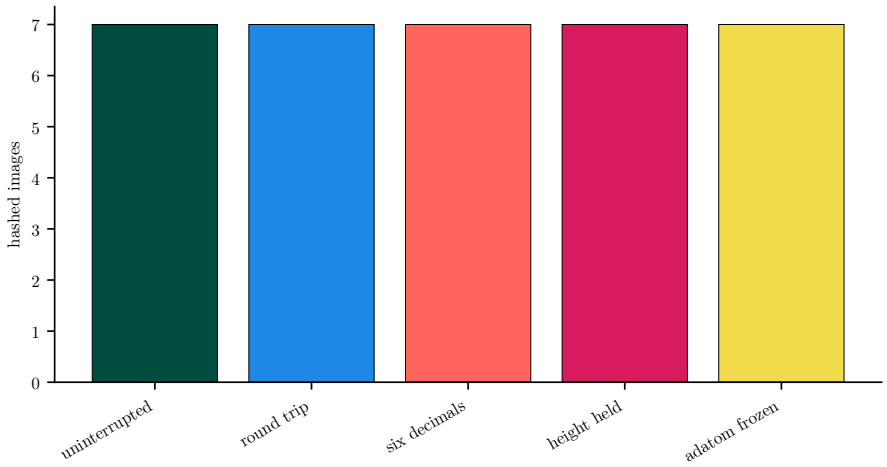


Figure 25: Hashed images in each stored campaign run.

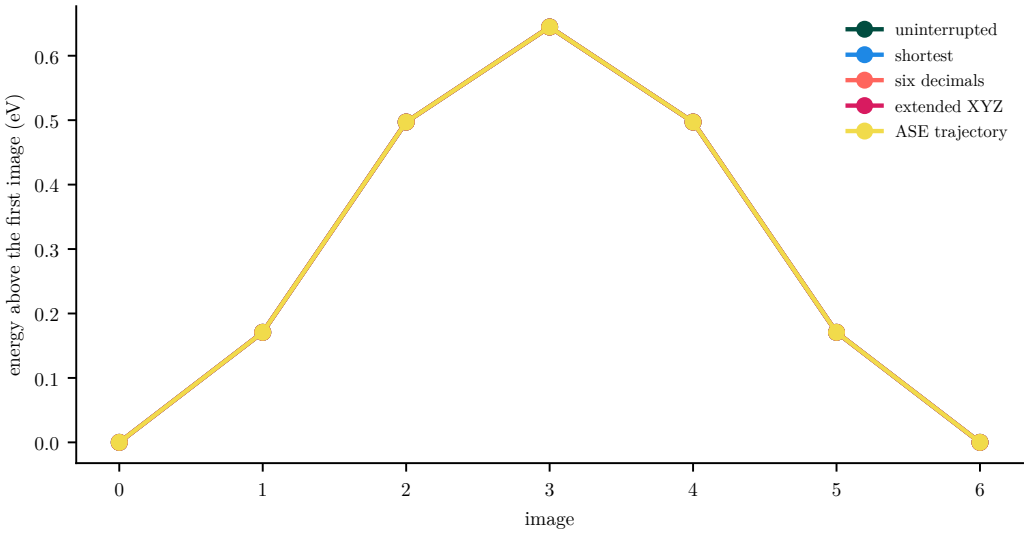


Figure 26: The five stored EMT bands, as energy above the first image.

11. Conclusions

A resumed saddle search repeats the interrupted one only when its checkpoint returns the fixed components, the identifiers, the time unit and the binary64 coordinates as written. CON 3.1 states these fields as a versioned grammar for the CON format, which many codes already write and read. readcon-core is the reference implementation of that grammar, in Rust, with C, C++, Python, Julia and Fortran interfaces. Every read path of the readcon-core Python module and C library returns the verdict of the corpus manifests on all 165 legal and illegal files. On the held Pt(100) slab, a band resumed from CON 3.1 repeats the uninterrupted band bit for bit, and a reader that drops the hold sends the band to a saddle with a prefactor 37 times larger. The reads measured on this branch are in Section 9. A format that defines no place for the mask cannot return that field. Unlike a format whose meaning lives in the code of one reader, CON 3.1 lets any program test its reader against the corpus, add a section under a later minor without breaking older readers, and resume a search from the files of another code.

Data availability

The software described here is readcon-core 0.18.1. The library measured here is the untagged commit

`f2cea7e88390cd6d295e0f455d8faf2a3fcdc68e`

The public main branch is

`df7ae1d19b05f2dc1a2ee4b1be0cfa7006f08aff`

The measured commit is not on that branch. `readcon-core.bundle` is a complete bundle of branch paper/pipeline-v13. `git clone readcon-core.bundle` needs no other commit. `readcon-library.bundle` contains one commit whose parent is the public base above. The library files of that commit have digest `d94b6ad922e9`. That bundle does not contain the measured commit. The latest public release tag is v0.18.0 at

`d5e77f997438557c72c3b3e360d446310f85d6b7`

No tag names this commit. The latest release tag is v0.18.0. The reader exchange in `paper/cpc/logs/reader-exchange.log` was measured on the clean

commit `e286bf570120069cbdf221d14219a381dd3eda2a`. The stored campaigns in Table S18, including r0174 and the head-to-head records, were not rerun on that commit. Each of those records names the library it was measured with. Listing S1 clones `readcon-core.bundle`. Rebuild the library from the untagged commit above with `cargo build --release`. The concept record of readcon-core [3] is

https://doi.org/10.5281/zenodo.18753567

The version DOI for this untagged commit is not minted.

Two commits in this repository carry parts of the reader used here: `458001d` gives the Python binding one periodic flag per cell vector, and `593f837` (pull request 72) adds the frame index. In a checkout of that untagged commit, `cargo build --release` builds the Rust library, the shared and static C libraries and the command-line program `readcon-core`, and `--features zstd,parallel` adds the two features. `meson setup build` and `meson compile -C build` build the C libraries with their headers and a pkg-config file, and `python3 -m pip install .` builds and installs the Python module with both features. Cargo, pip and conda fetch the dependencies that `Cargo.lock` and `paper/cpc/pixi.lock` pin, and the getting-started page of the documentation lists the CMake, Julia and Fortran commands [70]. readcon-core is distributed under the Massachusetts Institute of Technology (MIT) licence, and `addl/referenceImpls/eon_cpp` contains the CON reader of eOn under the BSD 3-clause licence of the eOn Development Team, which no build uses. The library of a commit is its files `src`, `include`, `build.rs`, `Cargo.toml`, `Cargo.lock` and `pyproject.toml`. Its digest, the SHA-256 over their `sha256sum` lines in byte order of their paths, begins `d94b6ad922e9` for this checkout. This checkout rejects a cell whose radicand lies outside the rounding window of `cell_vectors`, and a declared atom count larger than the bytes that remain. The reader exchange names the clean commit `e286bf570120069cbdf221d1 4219a381dd3eda2a`. The other timing records name the library they were measured with, `d94b6ad922e9`. The scripts, records and sources of this paper are the directory `paper/cpc`. The scripts in `paper/cpc/scripts` regenerate every table, figure and printed number from the records, each generated table names its script, and `paper/cpc/data/trace/numbers.csv` names the record and the rule behind each number printed in the text and the captions. The specification and the conformance corpus are `docs/orgmode/spec.org` and `resources/conformance` of that commit, the CON 3.1 corpus is `paper/cpc/data/corpus31`, and the timing records of this branch are in `paper/cpc/logs`. Section S11 gives the

commands that install readcon-core, rebuild the paper with TeX Live or Tectonic and rerun each measurement, and Table S18 maps every result to the release, the commit, the compiler and the host it was measured on.

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Visualization, Writing – original draft, Writing – review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The author declares no competing interests.

Acknowledgements

The author thanks his family, Amrita Goswami, Ruhila Goswami, Sonaly Goswami, Debabrata Goswami, Moritz Sallermann, and Nagaselvamani and Sankaranarayan, for their love and patience. Ruhila Goswami designed the colour palette used for every figure. He thanks Arí, Yoda, Jude, Crystee, Thor, Loki and the garden cats for their company, and TuiTui and Snoopy in memoriam. Dedicated, with love, to the memory of Cosmo, a baby bird. He also thanks Ami the cactus.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

Chatbots were used to refine certain turns of phrase and to speed through some implementation details; all the work is the author's own. The author reviewed and edited all tool output as needed and takes full responsibility for the content of the published article.

References

[1] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.

[2] R. Goswami, S. Chill, R. Terrell, G. Henkelman, M. Welborn, L. Zhang, A. Pedersen, E. Edelmann, S. H. Jung, S. A. Ghasemi, T. Dijkhuis, et al., TheochemUI/eOn: v3.6.0, Zenodo, software, version 3.6.0 (2026). doi:10.5281/zenodo.23166734.

[3] R. Goswami, readcon-core: a CON/convel reader/writer with FFI bindings, Zenodo, software, all versions (2026). doi:10.5281/zenodo.18753567.

[4] R. Goswami, readcon-core: a con/convel reader/writer with ffi bindings, concept DOI for the readcon-core software archive. Freeze tag v0.14.9. (2026). doi:10.5281/zenodo.18753567.
URL https://doi.org/10.5281/zenodo.18753567

[5] H. Eyring, The activated complex in chemical reactions, The Journal of Chemical Physics 3 (2) (1935) 107–115. doi:10.1063/1.1749604.

[6] G. H. Vineyard, Frequency factors and isotope effects in solid state rate processes, Journal of Physics and Chemistry of Solids 3 (1-2) (1957) 121–127. doi:10.1016/0022-3697(57)90059-8.

[7] P. Hänggi, P. Talkner, M. Borkovec, Reaction-rate theory: fifty years after Kramers, Reviews of Modern Physics 62 (2) (1990) 251–341. doi:10.1103/revmodphys.62.251.

[8] H. Jónsson, G. Mills, K. W. Jacobsen, Nudged elastic band method for finding minimum energy paths of transitions, in: Classical and Quantum Dynamics in Condensed Phase Simulations, WORLD SCIENTIFIC, 1998, pp. 385–404. doi:10.1142/9789812839664_0016.

[9] G. Henkelman, B. Uberuaga, H. Jónsson, A climbing image nudged elastic band method for finding saddle points and minimum energy

paths, The Journal of Chemical Physics 113 (22) (2000) 9901–9904. doi:10.1063/1.1329672.

[10] G. Henkelman, H. Jónsson, Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points, The Journal of Chemical Physics 113 (22) (2000) 9978–9985. doi:10.1063/1.1323224.

[11] V. Ásgeirsson, B. O. Birgisson, R. Bjornsson, U. Becker, F. Neese, C. Riplinger, H. Jónsson, Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following, Journal of Chemical Theory and Computation 17 (8) (2021) 4929–4945. doi:10.1021/acs.jctc.1c00462.

[12] G. Henkelman, H. Jónsson, A dimer method for finding saddle points on high dimensional potential surfaces using only first derivatives, The Journal of Chemical Physics 111 (15) (1999) 7010–7022. doi:10.1063/1.480097.

[13] J. Kästner, P. Sherwood, Superlinearly converging dimer method for transition state search, The Journal of Chemical Physics 128 (1) (2008) 014106. doi:10.1063/1.2815812.

[14] R. A. Olsen, G. J. Kroes, G. Henkelman, A. Arnaldsson, H. Jónsson, Comparison of methods for finding saddle points without knowledge of the final states, The Journal of Chemical Physics 121 (20) (2004) 9776–9792. doi:10.1063/1.1809574.

[15] W. E, W. Ren, E. Vanden-Eijnden, String method for the study of rare events, Physical Review B 66 (5) (2002) 052301. doi:10.1103/physrevb.66.052301.

[16] B. Peters, A. Heyden, A. T. Bell, A. Chakraborty, A growing string method for determining transition states: Comparison to the nudged elastic band and string methods, The Journal of Chemical Physics 120 (17) (2004) 7877–7886. doi:10.1063/1.1691018.

[17] G. Henkelman, H. Jónsson, Long time scale kinetic Monte Carlo simulations without lattice approximation and predefined event table, The Journal of Chemical Physics 115 (21) (2001) 9657–9666. doi:10.1063/1.1415500.

[18] L. Xu, G. Henkelman, Adaptive kinetic Monte Carlo for first-principles accelerated dynamics, The Journal of Chemical Physics 129 (11) (2008) 114104. doi:10.1063/1.2976010.

[19] A. F. Voter, F. Montalenti, T. C. Germann, Extending the time scale in atomistic simulation of materials, Annual Review of Materials Research 32 (1) (2002) 321–346. doi:10.1146/annurev.matsci.32.112601.141541.

[20] D. Perez, E. D. Cubuk, A. Waterland, E. Kaxiras, A. F. Voter, Long-time dynamics through parallel trajectory splicing, Journal of Chemical Theory and Computation 12 (1) (2016) 18–28. doi:10.1021/acs.jctc.5b00916.

[21] D. P. Anderson, BOINC: A system for public-resource computing and storage, in: Fifth IEEE/ACM International Workshop on Grid Computing, IEEE, 2004, pp. 4–10. doi:10.1109/grid.2004.14.

[22] A. Pedersen, H. Jónsson, Distributed implementation of the adaptive kinetic Monte Carlo method, Mathematics and Computers in Simulation 80 (7) (2010) 1487–1498. doi:10.1016/j.matcom.2009.02.010.

[23] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648x/aa680e.

[24] E. D. Hermes, K. Sargsyan, H. N. Najm, J. Zádor, Sella, an open-source automation-friendly molecular saddle point optimizer, Journal of Chemical Theory and Computation 18 (11) (2022) 6974–6988. doi:10.1021/acs.jctc.2c00395.

[25] G. Kresse, J. Furthmüller, Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set,

Computational Materials Science 6 (1) (1996) 15–50.
doi:10.1016/0927-0256(96)00008-0.

[26] IEEE Computer Society, IEEE standard for floating-point arithmetic (2019). doi:10.1109/ieeestd.2019.8766229.

[27] G. L. Steele, Jr., J. L. White, How to print floating-point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 112–126. doi:10.1145/93542.93559.

[28] U. Adams, Ryū: fast float-to-string conversion, in: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2018, pp. 270–282. doi:10.1145/3192366.3192369.

[29] W. D. Clinger, How to read floating point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation – PLDI '90, ACM Press, 1990, pp. 92–101. doi:10.1145/93542.93557.

[30] J. Kermode, libAtoms/extxyz: v0.4.6, extended XYZ specification and parsing tools, GitHub, software and format specification, version 0.4.6 (2026).
URL https://github.com/libAtoms/extxyz/releases/tag/v0.4.6

[31] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648X/aa680e.

[32] P. de Buyl, P. H. Colberg, F. Höfling, H5MD: A structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.

[33] P. de Buyl, P. H. Colberg, F. Höfling, H5MD: A structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.

[34] M. Folk, G. Heber, Q. Koziol, E. Pourmal, D. Robinson, An overview of the HDF5 technology suite and its applications, in: Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases, ACM, 2011, pp. 36–47. doi:10.1145/1966895.1966900.

[35] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, SoftwareX 1-2 (2015) 19–25. doi:10.1016/j.softx.2015.06.001.

[36] B. R. Brooks, C. L. Brooks, A. D. Mackerell, L. Nilsson, R. J. Petrella, B. Roux, Y. Won, G. Archontis, C. Bartels, S. Boresch, A. Caflisch, L. Caves, Q. Cui, A. R. Dinner, M. Feig, S. Fischer, J. Gao, M. Hodoscek, W. Im, K. Kuczera, T. Lazaridis, J. Ma, V. Ovchinnikov, E. Paci, R. W. Pastor, C. B. Post, J. Z. Pu, M. Schaefer, B. Tidor, R. M. Venable, H. L. Woodcock, X. Wu, W. Yang, D. M. York, M. Karplus, CHARMM: The biomolecular simulation program, Journal of Computational Chemistry 30 (10) (2009) 1545–1614. doi:10.1002/jcc.21287.

[37] D. Spångberg, D. S. D. Larsson, D. van der Spoel, Trajectory NG: portable, compressed, general molecular dynamics trajectories, Journal of Molecular Modeling 17 (10) (2011) 2669–2685. doi:10.1007/s00894-010-0948-5.

[38] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2014) 260–269. doi:10.1002/jcc.23495.

[39] VASP Software GmbH, POSCAR, VASP Wiki, accessed 2026-10-30 (2026).
URL https://www.vasp.at/wiki/index.php/POSCAR

[40] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F. Coudert, M. Levesque, S. Maiti, S. Guionniere, et al., chemfiles/chemfiles: Version 0.10.4, Zenodo, software, version 0.10.4 (2023). doi:10.5281/zenodo.7904565.

[41] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F. Coudert, pelsa, M. Levesque, S. Maiti, S. Guionniere, jmintser, chemfiles (2023). doi:10.5281/zenodo.1202207.
URL https://doi.org/10.5281/zenodo.1202207

[42] R. Gowers, M. Linke, J. Barnoud, T. Reddy, M. Melo, S. Seyler, J. Domański, D. Dotson, S. Buchoux, I. Kenney, O. Beckstein, MDAnalysis: A Python package for the rapid analysis of molecular dynamics simulations, in: Proceedings of the Python in Science Conference, SciPy, 2016, pp. 98–105. doi:10.25080/majora-629e541a-00e.

[43] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, Journal of Computational Chemistry 32 (10) (2011) 2319–2327. doi:10.1002/jcc.21787.

[44] A. Stukowski, Visualization and analysis of atomistic simulation data with OVITO–the Open Visualization Tool, Modelling and Simulation in Materials Science and Engineering 18 (1) (2010) 015012. doi:10.1088/0965-0393/18/1/015012.

[45] N. M. O'Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, G. R. Hutchison, Open Babel: An open chemical toolbox, Journal of Cheminformatics 3 (1) (2011) 33. doi:10.1186/1758-2946-3-33.

[46] J. Alwall, A. Ballestrero, P. Bartalini, S. Belov, E. Boos, A. Buckley, J. M. Butterworth, L. Dudko, S. Frixione, L. Garren, S. Gieseke, A. Gusev, I. Hinchliffe, J. Huston, B. Kersevan, F. Krauss, N. Lavesson, L. Lönnblad, E. Maina, F. Maltoni, M. L. Mangano, F. Moortgat, S. Mrenna, C. G. Papadopoulos, R. Pittau, P. Richardson, M. H. Seymour, A. Sherstnev, T. Sjöstrand, P. Skands, S. R. Slabospitsky, Z. Was, B. R. Webber, M. Worek, D. Zeppenfeld, A standard format for Les Houches Event Files, Computer Physics Communications 176 (4) (2007) 300–304. doi:10.1016/j.cpc.2006.11.010.

[47] B. C. Allanach, C. Balázs, G. Bélanger, M. Bernhardt, F. Boudjema, D. Choudhury, K. Desch, U. Ellwanger, P. Gambino, R. Godbole, T. Goto, J. Guasch, M. Guchait, T. Hahn, S. Heinemeyer, C. Hugonie, T. Hurth, S. Kraml, S. Kreiss, J. Lykken, F. Moortgat, S. Moretti, S. Peñaranda, T. Plehn, W. Porod, A. Pukhov, P. Richardson, M. Schumacher, L. Silvestrini, P. Skands, P. Slavich, M. Spira, G. Weiglein, P. Wienemann, SUSY Les Houches Accord 2, Computer Physics Communications 180 (1) (2009) 8–25. doi:10.1016/j.cpc.2008.08.004.

[48] A. Buckley, P. Ilten, D. Konstantinov, L. Lönnblad, J. Monk, W. Pokorski, T. Przedzinski, A. Verbytskyi, The HepMC3 event record library for Monte Carlo event generators, Computer Physics Communications 260 (2021) 107310. doi:10.1016/j.cpc.2020.107310.

[49] X. Gonze, C.-O. Almbladh, A. Cucca, D. Caliste, C. Freysoldt, M. A. L. Marques, V. Olevano, Y. Pouillon, M. J. Verstraete, Specification of an extensible and portable file format for electronic structure and crystallographic data, Computational Materials Science 43 (4) (2008) 1056–1065. doi:10.1016/j.commatsci.2008.02.023.

[50] E. Posenitskiy, V. G. Chilkuri, A. Ammar, M. Hapka, K. Pernal, R. Shinde, E. J. Landinez Borda, C. Filippi, K. Nakano, O. Kohulák, S. Sorella, P. de Oliveira Castro, W. Jalby, P. L. Ríos, A. Alavi, A. Scemama, TREXIO: A file format and library for quantum chemistry, The Journal of Chemical Physics 158 (17) (2023) 174801. doi:10.1063/5.0148161.

[51] A. García, M. J. Verstraete, Y. Pouillon, J. Junquera, The psml format and library for norm-conserving pseudopotential data curation and interoperability, Computer Physics Communications 227 (2018) 51–71. doi:10.1016/j.cpc.2018.02.011.

[52] T. Bray, The JavaScript object notation (JSON) data interchange format, Tech. Rep. 8259, IETF (2017). doi:10.17487/rfc8259.

[53] H. Bekker, Unification of box shapes in molecular simulations, Journal of Computational Chemistry 18 (15) (1997) 1930–1942. doi:10.1002/(sici)1096-987x(19971130)18:15<1930::aid-jcc8>3.0.co;2-p.

[54] E. Tiesinga, P. J. Mohr, D. B. Newell, B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, Reviews of Modern Physics 93 (2) (2021) 025010. doi:10.1103/revmodphys.93.025010.

[55] D. Lemire, Number parsing at a gigabyte per second, Software: Practice and Experience 51 (8) (2021) 1700–1727. doi:10.1002/spe.2984.

[56] B. Ford, Parsing expression grammars: a recognition-based syntactic foundation, in: Proceedings of the 31st ACM SIGPLAN-SIGACT symposium on Principles of programming languages, ACM, 2004, pp. 111–122. doi:10.1145/964001.964011.

[57] F. Loitsch, Printing floating-point numbers quickly and accurately with integers, in: Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation, ACM, 2010, pp. 233–243. doi:10.1145/1806596.1806623.

[58] M. Andrysco, R. Jhala, S. Lerner, Printing floating-point numbers: a faster, always correct method, in: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM, 2016, pp. 555–567. doi:10.1145/2837614.2837654.

[59] P. Deutsch, GZIP file format specification version 4.3, Tech. Rep. 1952, IETF (1996). doi:10.17487/rfc1952.

[60] Y. Collet, Zstandard compression and the 'application/zstd' media type, Tech. Rep. 8878, IETF (2021). doi:10.17487/rfc8878.

[61] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.

[62] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[63] DLPack contributors, DLPack: open in-memory tensor structure, tensor exchange consumed by NumPy, PyTorch, JAX and CuPy (2024). URL https://github.com/dmlc/dlpack

[64] R. Gowers, O. Beckstein, M. Linke, I. Alibay, J. Barnoud, T. Reddy, H. MacDermott-Opeskin, R. Meli, L. Wang, U. Bansal, J. Zeman, J. Detlefs, D. L. Dotson, M. N. Melo, M. Bieniek, S. Buchoux, M. Tiberti, N. Michaud-Agrawal, D. Cruz, S. Seyler, P. Loche, J. Borreguero, Y. Zhuang, T. Yelgoe, N. Pal, J. Domański, Z. Wu, et al., MDAnalysis/mdanalysis: Release 2.10.0 of MDAnalysis and MDAnalysisTests, Zenodo, software, version package-2.10.0 (2025). doi:10.5281/zenodo.17382469.

[65] K. W. Jacobsen, P. Stoltze, J. K. Nørskov, A semi-empirical effective medium theory for metals and alloys, Surface Science 366 (2) (1996) 394–402. doi:10.1016/0039-6028(96)00816-3.

[66] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, P. Gumbsch, Structural relaxation made simple, Physical Review Letters 97 (17) (2006) 170201. doi:10.1103/physrevlett.97.170201.

[67] J. Nocedal, Updating quasi-Newton matrices with limited storage, Mathematics of Computation 35 (151) (1980) 773–782. doi:10.1090/s0025-5718-1980-0572855-7.

[68] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for finding minimum energy paths, The Journal of Chemical Physics 128 (13) (2008) 134106. doi:10.1063/1.2841941.

[69] W. C. Swope, H. C. Andersen, P. H. Berens, K. R. Wilson, A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters, The Journal of Chemical Physics 76 (1) (1982) 637–649. doi:10.1063/1.442716.

[70] R. Goswami, readcon-core documentation: Getting started, https://lode-org.github.io/readcon-core/getting-started.html, accessed 2026-11-01 (2026).